

剑指 LLM 预训练 算法岗面试 100 题

从 Transformer 八股，走到数据、Scaling、分布式、MoE、长上下文与训练事故现场

定位

面向 LLM / Foundation Model 预训练算法岗。按“题目 → Why → 主答 → Know-how → 追问 → 失分点 → 一句话记忆”组织，训练的是 3 分钟主答 + 10 分钟连续追问能力。

结构说明：借鉴《剑指 Offer》“问题驱动、层层追问、重视思路”的学习范式，但内容、题解和版式均为独立编写，不复制原书。

第一版 • 2026 年 9 月

技术校准：Qwen3 / DeepSeek-V3 / Chinchilla / FlashAttention / Megatron Core

使用方法：把 100 题刷成一套可迁移框架

预训练面试的核心不是记住更多名词，而是面对任意问题都能沿同一条逻辑链展开：What → Why → How → Trade-off → Production。

层级	面试要回答什么	示例：FlashAttention
What	定义与公式	Exact dense attention。
Why	旧方案瓶颈	$S \times S$ 中间矩阵造成大量 HBM 读写。
How	如何实现	Tiling、SRAM reuse、online softmax。
Trade-off	代价与边界	数学 FLOPs 仍是 $O(S^2d)$ ，实现复杂。
Production	集群里怎么验证	看 HBM traffic、kernel time、端到端 tokens/s。

四轮刷题法

- 1. 只看题目，3 分钟闭卷主答。
- 2. 只练公式、Tensor Shape、显存与通信手算。
- 3. 用“高频追问”自我追杀，每题至少连续回答 3 个 Why。
- 4. 映射到自己的真实项目：把模型大小、Token、GPU、并行度、MFU、故障案例说成具体数字。

核心原则：如果一个答案无法落到“我会怎样验证它”，说明仍停留在知识层；预训练岗位最终看实验与系统闭环。

知识地图与目录

100 题分为十章。第 31-60 与 81-100 是区分“普通 LLM 算法岗”和“预训练核心岗”的高价值区域。

第 1 章 · 题 01-10

Transformer 与模型架构

从公式、表示能力、参数量与硬件友好性四条线理解现代 Decoder-only。

第 2 章 · 题 11-20

Tokenizer 与预训练数据

数据决定上限：清洗、去重、配比、污染控制、合成数据与 Packing。

第 3 章 · 题 21-30

目标函数、优化器与训练配方

从 next-token prediction 到 AdamW、Warmup、Batch Token 与阶段化训练。

第 4 章 · 题 31-40

Scaling Law 与算力预算

把参数规模、训练 Token、FLOPs、GPU-hours 与推理成本统一起来。

第 5 章 · 题 41-50

分布式训练

DP / TP / PP / SP / CP / EP / FSDP：切什么、怎么通信、为什么这样组合。

第 6 章 · 题 51-60

显存、精度与训练性能

会手算显存，会解释 BF16/FP8，会区分 compute wall、memory wall 与 communication wall。

第 7 章 · 题 61-70

Mixture-of-Experts

从稀疏激活到路由、负载均衡、专家并行与 All-to-All。

第 8 章 · 题 71-80

长上下文与高效 Attention

RoPE 外推、FlashAttention、Sliding Window、Context Parallel 与有效上下文评测。

第 9 章 · 题 81-90

训练稳定性、Debug 与评测

真正做过预训练的人，必须能定位 Loss Spike、NaN、Straggler、数据异常与恢复问题。

第 10 章 · 题 91-100

手撕、系统设计与项目拷打

把“知道”变成“能写、能算、能设计、能复盘、能证明”。

20 道 S 级必练：01、07、14、17、19、27、31、34、35、42、46、48、50、52、58、63、65、81、96、100。

第 1 章 · 题 01-10

Transformer 与模型架构

从公式、表示能力、参数量与硬件友好性四条线理解现代 Decoder-only。

本章面试策略

先把核心公式 / Shape 写出来，再解释设计动机；最后一定落到显存、通信、吞吐或数据分布。

题 01 手推 Self-Attention：为什么要除以 $\sqrt{d_k}$ ？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

检验你是否真正理解注意力的数值尺度，而不是只会背 API。

3 分钟主答

给定 X ，先做线性投影得到 $Q=XW_Q$ 、 $K=XW_K$ 、 $V=XW_V$ ，再计算缩放点积注意力。若 q_i 、 k_i 近似独立、零均值、单位方差，则 $q \cdot k$ 的方差随 d_k 线性增长；维度越大，softmax 输入越容易进入饱和区。

除以 $\sqrt{d_k}$ 相当于把 logits 的标准差重新压到 $O(1)$ 量级，使 softmax 的熵和梯度尺度更稳定。它不是“为了让概率和为 1”——softmax 本身已经完成归一化。

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Know-how：落到实现 / 训练系统

- 白板先写 $Q/K/V$ 与 tensor shape，再解释方差尺度。
- 训练中同时监控 attention logits 的 max/RMS 与梯度范数；异常放大常与数值不稳定相关。
- 进一步联系 QK-Norm：它直接约束 Q/K 的范数，是更强的 logit-scale 稳定手段。

深挖关键点 优秀答案会把“数学尺度 $\rightarrow$ softmax 饱和 $\rightarrow$ 训练可观测指标”连成一条链。

高频追问

1. 为什么不是除以 d_k ？
2. 如果 head dimension 从 64 改到 256，会发生什么？
3. BF16 下 attention logit 过大时你会看哪些指标？

常见失分点

- 只说“防止梯度消失”而不给方差推导。
- 把 scaling 说成 softmax 概率归一化。

一句话记忆 $\sqrt{d_k}$ 的本质是标准差归一化：让点积 logits 的典型尺度不随 head dimension 爆长。

延伸阅读：R1

题 02 MHA、MQA、GQA 的本质区别是什么？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

考察你能否把表示能力、KV Cache、内存带宽和训练架构统一起来。

3 分钟主答

MHA 为每个 query head 配一组独立 K/V ；MQA 让所有 query heads 共享一组 K/V ；GQA 介于两者之间，让若干 query heads 共享一个 KV head。若 h_q 为 query heads、 h_{kv} 为 KV heads，则 MHA: $h_{kv}=h_q$ ，MQA: $h_{kv}=1$ ，GQA: $1 < h_{kv} < h_q$ 。

推理时 KV Cache 大小近似正比于 h_{kv} ，因此减少 KV heads 能显著降低显存占用和内存带宽；代价是 K/V 表示自由度下降。GQA 是质量与 serving efficiency 的折中。

Know-how: 落到实现 / 训练系统

- 手算 KV Cache 时显式写 batch、sequence、layers、 $2(K/V)$ 、 h_{kv} 、head_dim、dtype。
- 实现中注意 repeat_kv 只是逻辑广播；不应真的复制成 MHA 大小再做 attention。
- 训练架构选型要结合预期推理吞吐，而不是只看 pretraining loss。

深挖关键点 把架构题回答成全生命周期成本题，会明显体现预训练岗位的系统意识。

高频追问

1. GQA 为什么不只是一种推理优化？
2. GQA 的参数量比 MHA 少多少？
3. 为什么 KV heads 太少可能伤质量？

常见失分点

- 只回答“GQA 更省显存”，不说明省的是 KV Cache 哪一维。
- 把 query heads 也错误地一起减少。

一句话记忆 GQA 的关键不是“少几个头”，而是用更少的 KV 表示换取更低的缓存与带宽成本。

延伸阅读：R3 / R15

题 03 为什么大模型主流选择 Decoder-only?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

检验你能否从目标函数、数据可扩展性和推理形态解释架构趋势。

3 分钟主答

Decoder-only 使用统一的 causal language modeling：任意文本都可转换为 next-token prediction，无需为不同任务设计专门目标；训练目标和自回归生成形式也天然一致。

它的优势不是“理论上永远优于 Encoder/Encoder-Decoder”，而是数据来源最广、scale-up 路径简单、in-context learning 兼容性好、工程栈统一。对纯表示学习或强条件生成任务，其他结构仍可能更合适。

Know-how: 落到实现 / 训练系统

- 回答时先分任务：生成、理解、seq2seq，再谈为何 foundation model 偏向统一生成接口。
- 指出 causal mask 带来的训练并行与推理串行差异。
- 不要把产业选择误写成架构定理。

高频追问

1. BERT 为什么仍适合 embedding？

常见失分点

- 只说“GPT 成功所以大家都用”。
- 宣称 Decoder-only 在所有任务都更强。

2. Encoder-Decoder 在翻译/多模态中还有什么优势？
3. Decoder-only 的主要训练与推理瓶颈是什么？

一句话记忆 Decoder-only 赢在“统一目标 + 海量无标注数据 + 统一生成接口”的可扩展性，而不是结构上绝对支配。

延伸阅读：R1

题 04 Pre-Norm 与 Post-Norm：为什么超深模型常用 Pre-Norm？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察残差路径、梯度传播和深层网络稳定性。

3 分钟主答

Post-Norm 常写作 $y = \text{LN}(x + F(x))$ ；Pre-Norm 写作 $y = x + F(\text{LN}(x))$ 。Pre-Norm 中 residual stream 提供更直接的 identity path，深层堆叠时梯度传播更稳，通常能减少训练初期发散风险。

但“更稳”不等于“表达能力必然更好”。不同模型还会使用 RMSNorm、sandwich norm、residual scaling 等改造，因此回答应落到优化稳定性而非教条。

Know-how：落到实现 / 训练系统

- 画出残差分支并说明 backward 的直接路径。
- 观察 per-layer activation RMS 与 gradient norm，判断深层是否出现尺度漂移。
- 做小模型 depth scaling 实验，而不是只对比单一规模。

高频追问

1. Post-Norm 为什么可能有更强的层间变换？
2. Pre-Norm 是否会导致 residual stream 变大？
3. RMSNorm 与 Pre-Norm 是同一个维度的问题吗？

常见失分点

- 把 Norm 位置与 Norm 类型混为一谈。
- 只背“Pre-Norm 稳定”，无法解释 identity path。

一句话记忆 Pre-Norm 的主要价值是给深网络保留一条更干净的残差/梯度高速公路。

延伸阅读：R1

题 05 RMSNorm 与 LayerNorm 的区别是什么？

A 类高频真题型

难度 易

频率 ★★★★★★

面试官为什么问

高频基础题，同时能追到数值稳定、参数和 kernel 实现。

3 分钟主答

LayerNorm 对特征做去均值再除以标准差；RMSNorm 不减均值，只按均方根缩放。 $\text{RMSNorm}(x) = x / \sqrt{(\text{mean}(x^2) + \epsilon)}$ 。

RMSNorm 省掉 mean-centering，计算路径更简单，在现代 LLM 中被大量采用。关键不是“少一个减法”，而是很多 Transformer residual stream 并不依赖显式去均值也能稳定训练。

$$\text{RMS}(x) = \sqrt{(\text{mean}(x^2) + \epsilon)}, y = x / \text{RMS}(x) \cdot \gamma$$

Know-how：落到实现 / 训练系统

- 手写时使用 $\text{mean}(x^2)$ ，不能误用 std 。
- 归一化通常用更高精度累积，尤其在低精度训练中。
- 检查 epsilon、weight dtype、fused kernel 与张量布局。

高频追问

1. RMSNorm 为什么没有 β ？
2. epsilon 过大/过小会怎样？
3. 为什么 normalization 常保留 FP32 accumulation？

常见失分点

- 把 RMS 写成标准差。
- 只讲速度，不讲去均值这一统计差异。

一句话记忆 RMSNorm 保留“控制尺度”，放弃“强制零均值”。

题 06 SwiGLU 为什么替代传统 ReLU/GELU FFN？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察 gated FFN 的表达能力与参数预算。

3 分钟主答

SwiGLU 使用两条输入投影：一条经过 SiLU 形成 gate，另一条形成 value，二者逐元素相乘后再投影回 hidden dimension。门控让网络可以根据输入选择性调制特征。

因为多了一组输入投影，若仍使用传统 4d intermediate dimension，参数与 FLOPs 会显著变大；实际架构常调整 intermediate size，使总预算与目标参数规模匹配。

Know-how：落到实现 / 训练系统

- 参数手算时分别计 gate/up/down 三个矩阵。
- 比较 FFN 的 GEMM shape 是否硬件友好，不能只按参数最优。

- 关注激活分布和 fused SwiGLU kernel。

高频追问

1. SwiGLU 参数量怎么计算？
2. 为什么用 SiLU 而不是 sigmoid？
3. intermediate dimension 如何按预算反推？

常见失分点

- 把 SwiGLU 说成 “把 GELU 换成 SiLU”。
- 忘记 gate 和 up 是两套权重。

一句话记忆 SwiGLU 的关键词是 gating；参数预算必须按三块 projection 重新计算。

延伸阅读：R4

题 07 RoPE 为什么能把相对位置信息写进 Attention？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

这是最常见的 “会不会数学推导” 分界题。

3 分钟主答

RoPE 将每对隐藏维度视为二维平面，根据 token position 做旋转。位置 m 的 q 变成 $R_m q$ ，位置 n 的 k 变成 $R_n k$ 。由于旋转矩阵满足 $R_m^T R_n = R_{\{n-m\}}$ ，二者点积只通过相对位移 $n-m$ 进入。

因此 RoPE 同时保留绝对位置施加在向量上的变换，又让 QK 内积天然具有相对位置结构。这比只说 “旋转编码” 更完整。

$$(R_m q)^T (R_n k) = q^T R_{\{n-m\}} k$$

Know-how：落到实现 / 训练系统

- 白板至少推到 $q^T R_{\{n-m\}} k$ 。
- 实现时检查偶/奇维配对、频率表、position_ids 与 broadcasting。
- 长上下文扩展时继续追问频率外推，而不是简单改 max_position_embeddings。

深挖关键点 这道题应练到能在 60 秒内从二维旋转矩阵推到相对位移。

高频追问

1. 为什么 $R_m^T R_n = R_{\{n-m\}}$ ？
2. RoPE 为什么会有外推问题？
3. 不同频率维度分别偏向什么距离尺度？

常见失分点

- 只背 cos/sin 代码，没有相对位置推导。
- 把 RoPE 说成可学习位置 embedding。

一句话记忆 RoPE 最关键的一行：旋转后的 QK 点积由 $R_{\{n-m\}}$ 决定。

延伸阅读：R2

题 08 为什么现代 LLM 很少使用原始绝对位置 Embedding?

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察位置表示与长上下文扩展的设计取舍。

3 分钟主答

可学习 absolute embedding 直接为每个位置存向量，简单但把位置绑定到训练长度，跨长度泛化弱；固定 sinusoidal 能外推到新索引，但 Attention 的两两相对关系并未被显式编码。

RoPE、relative bias、ALiBi 等方法把相对关系更直接地作用到 attention score 或 Q/K 表示上，因此更适合长序列与跨长度泛化。但它们也都有自己的频率/偏置假设和外推边界。

Know-how：落到实现 / 训练系统

- 比较方法时统一从“参数量、相对关系、训练长度外推、kernel 兼容”四维回答。
- 不要把理论可外推等同于模型会利用长上下文。
- 长上下文能力必须靠训练数据和评测验证。

高频追问

1. Sinusoidal 为什么理论上可表示任意位置？
2. ALiBi 与 RoPE 的 inductive bias 有何不同？
3. 位置编码会影响 KV Cache 吗？

常见失分点

- 把“支持更大 position id”当成“有效长上下文”。

一句话记忆 位置编码问题不是“有没有位置”，而是模型怎样表达相对距离并跨越训练长度泛化。

延伸阅读：R2

题 09 Embedding 与 LM Head 为什么要 Weight Tying? 什么时候不 Tying?

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察参数共享、表示空间和输出层预算。

3 分钟主答

Weight tying 令输出 projection 与 token embedding 共享权重，可显著减少 $V \times d$ 参数，并让输入和输出 token 表示处在同一空间。

不 tying 则提供更高的输出表示自由度，在大词表、特殊多模态 token、架构解耦等场景可能更灵活。面试应把它描述为 parameter efficiency 与 representation flexibility 的取舍。

Know-how：落到实现 / 训练系统

- 大词表模型里单独估算 embedding/LM head 占总参数比例。
- 检查 TP 下 vocab-parallel embedding 与 cross-entropy 的实现。
- 若扩词表，tying 会影响初始化和旧 token 权重保持策略。

高频追问

1. 大词表时 tying 节省多少参数？
2. 扩 vocabulary 时如何初始化新 token？
3. TP 下 vocab projection 怎么切？

常见失分点

- 说成“共享一定提升效果”。
- 忽略 vocab 很大时输出层也影响计算和通信。

一句话记忆 Weight tying 不是免费午餐：用参数共享换容量约束。

题 10 给你 7B 参数预算，如何从零设计 Decoder-only 架构？

B 类等价追问问题

难度 难

频率 ★★★★★

面试官为什么问

从单点知识升级到完整 architecture sizing，考察参数手算与硬件意识。

3 分钟主答

先明确约束：参数预算、目标上下文、训练硬件、推理形态与词表。Dense Transformer 的粗略 block 参数可从 Attention 与 FFN 估算，再决定 depth/width、head_dim、GQA heads、SwiGLU intermediate。

最优设计不是“复制 Llama”。层数/宽度影响优化与 GEMM shape；head_dim 影响 attention；GQA 影响 KV Cache；词表影响 embedding；TP 要求维度可整除。最终需要小规模 proxy runs 验证训练稳定与能力。

Know-how：落到实现 / 训练系统

- 先做参数 spreadsheet，再做单卡/多卡 kernel benchmark。
- 确保 hidden、heads、FFN dim、vocab size 对 TP 友好。
- 用多个宽深比例的小模型做等 FLOPs 对照，而非只训练一个配置。

深挖关键点 面试现场可先写 $P \approx L \cdot (P_{\text{attn}} + P_{\text{ffn}}) + P_{\text{embed}}$ ，再逐项补 GQA/SwiGLU 修正。

高频追问

1. 为什么更深不一定更好？
2. 如何反推 intermediate size？
3. 如果 serving KV Cache 是硬约束，你会改哪几项？

常见失分点

- 直接报某开源模型配置，没有推导。
- 只满足参数量，不看硬件矩阵形状与推理成本。

一句话记忆 架构 sizing 是“参数预算 × 优化稳定 × 硬件效率 × 推理成本”的联合设计。

延伸阅读：R1

第 2 章 · 题 11-20

Tokenizer 与预训练数据

数据决定上限：清洗、去重、配比、污染控制、合成数据与 Packing。

本章面试策略

不要把数据与训练配方当固定 cookbook；面试官更关心你如何用实验找到合适方案。

题 11 BPE、WordPiece、SentencePiece 的本质区别？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

Tokenizer 是数据工程与算力效率的第一层接口，不能只背名词。

3 分钟主答

BPE 通过迭代合并高频符号对构建词表；WordPiece 的经典实现更偏向按语言模型/似然收益选择子词；SentencePiece 是直接从原始字符串训练与编码的框架，可承载 BPE、Unigram 等算法。

对 LLM 来说，关键不是分类学，而是 tokenizer 如何改变序列长度、跨语言 fertility、数字/代码切分和词表参数。Tokenizer 设计会直接改变同一语料对应的训练 token 数。

Know-how：落到实现 / 训练系统

- 在目标语料上统计 chars/token、bytes/token、不同语言 fertility。
- 单独评估代码标识符、数字、小数、Unicode 与空白。
- 词表升级要考虑旧 checkpoint embedding 的兼容。

高频追问

1. SentencePiece 是一种具体算法吗？
2. Tokenizer 会怎样影响 32K context 的实际信息量？
3. 为什么 byte fallback 有价值？

常见失分点

- 只讲算法流程，不谈预训练 token 预算。

一句话记忆 Tokenizer 不是前处理小工具，它决定“每个训练 token 装了多少信息”。

题 12 Vocabulary Size 应该怎么定？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察序列长度、softmax、embedding 参数与长尾学习的平衡。

3 分钟主答

词表太小会把文本切得过碎，增加 sequence length，从而增加 attention/MLP 计算并降低有效上下文；词表太大则增加 embedding/LM head 参数和输出计算，同时大量 rare tokens 训练不足。

因此 V 不是越大越好，而应在目标语言、代码/数学、硬件吞吐和参数预算上做 Pareto 权衡。多语言模型还要防止某些语言被不公平地过度切分。

Know-how：落到实现 / 训练系统

- 对多个候选 vocab 真实 tokenize 训练语料，估算总 token 数变化。
- 把 V 对参数量与每 step tokens/sec 的影响一起测。
- 按语言/domain 报 fertility 分布，不只看总平均。

高频追问

1. 扩大 vocab 会让训练一定更快吗？
2. LM head 很大时怎样优化？
3. 多语言词表如何避免高资源语言挤占？

常见失分点

- 只根据“常见 32K/128K”拍脑袋。

一句话记忆 词表大小连接了语言压缩率与模型参数，是数据效率和硬件效率的共同超参。

题 13 什么是 Tokenizer Fertility? 为什么中文/多语言必须看？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

检验你是否会用可量化指标评估 tokenizer，而不是主观举例。

3 分钟主答

Fertility 可理解为一个自然语言单位平均被切成多少 token；工程上也常用 chars/token、bytes/token 等更稳定指标。若同一段中文在 tokenizer A 中需要 20 token、B 中需要 35 token，那么相同 32K token context 的实际信息容量差很多。

多语言预训练中，高 fertility 的语言需要更多 compute 才能看见同样文本量，也更容易在 fixed context 下被截断，因此 tokenizer 会隐式改变语言间的训练资源分配。

Know-how：落到实现 / 训练系统

- 按语言、脚本、domain 建 tokenization dashboard。
- 统计 p50/p95 而不只均值，识别异常文本。
- 把 fertility 与 validation loss / downstream capability 做相关分析。

高频追问

1. 如何定义中文的“一个自然语言单位”？
2. 为什么 bytes/token 常比 words/token 更可比？
3. Tokenizer 改了后 scaling law 还能直接比吗？

常见失分点

- 把 fertility 只解释成“词越长越好”。

一句话记忆 多语言 tokenizer 的公平性，本质上也是每种语言获得多少有效 compute 的问题。

题 14 从 Common Crawl 到训练 Token：完整预训练数据 Pipeline 怎么设计？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

这是预训练岗位最核心的 data-system 题，检验是否真正做过全链路。

3 分钟主答

典型链路包括：抓取/接入 → parsing 与正文抽取 → 语言识别 → 规则过滤 → quality scoring → 安全/PII 处理 → exact/near/semantic dedup → benchmark decontamination → domain tagging → mixture sampling → tokenization → packing → shuffle/sharding → dataloader。

关键是每一步都要可追溯、可回滚、可量化。不要把“清洗”当一个黑盒脚本；每个 filter 都会改变数据分布，必须记录保留率、domain/language 变化与质量收益。

Know-how：落到实现 / 训练系统

- 给每条样本保留 source_id、版本、过滤原因和 hash，支持数据 lineage。
- 建立 stage-by-stage retention dashboard，抽样做人审。
- 在 proxy model 上验证过滤/配比变化，而不是仅用 quality classifier 分数决策。

深挖关键点 把 pipeline 画成 DAG，并在每个节点标出输入规模、保留率、质量指标和版本号。

高频追问

1. 去重应该在 quality filtering 前还是后？
2. 如何做数据版本管理？
3. 怎样证明某个过滤规则真的提升了模型？

常见失分点

- 只列“清洗、去重、分词”三个词。
- 没有数据 lineage，出了 benchmark 污染无法追责。

一句话记忆 预训练数据工程的专业度体现在：每个 token 都能解释“从哪里来、为何留下、被采样多少次”。

延伸阅读：R11 / R12 / R13

题 15 Exact Dedup、MinHash、SimHash、Semantic Dedup 分别解决什么？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

去重既影响数据质量，也改变有效数据分布与记忆风险。

3 分钟主答

Exact dedup 用内容 hash 去除完全重复；MinHash+LSH 估计集合/Jaccard 相似度，适合 near-duplicate 文档或 shingles；SimHash 将特征投影到指纹空间，适合快速相似检索；semantic dedup 用 embedding/聚类处理表面不同但语义重复。

去重的价值不只是省存储。重复文档等价于给某些内容更高 sampling weight，会扭曲训练分布、增加 memorization，并浪费有限 compute。

Know-how：落到实现 / 训练系统

- 先做 URL/精确 hash，再做近重复可降低昂贵阶段的候选量。
- 按 document 与 span 两种粒度处理，避免整篇因局部模板被误删。
- 记录 cluster size，可用于重新加权而不是一律删除。

高频追问

1. MinHash 为什么近似 Jaccard？
2. 语义去重最危险的 false positive 是什么？
3. 代码数据如何去重？

常见失分点

- 认为“相似度越高删得越干净越好”。
- 不区分模板重复和核心内容重复。

一句话记忆 Dedup 本质是在修正隐式采样权重，而不仅是减少字节。

题 16 预训练数据为什么不能简单追求“质量越高越好”？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察 data diversity、coverage 与模型鲁棒性的统一认识。

3 分钟主答

单一“高质量”评分往往偏向规范、短、百科式文本；如果过度过滤，模型会失去对口语、长尾知识、真实网页噪声、不同风格和跨领域表达的覆盖。

预训练目标通常是 $\text{quality} \times \text{diversity} \times \text{coverage} \times \text{difficulty}$ 的组合。高质量数据提高信号密度，多样性保证世界覆盖，难例和噪声分布则关系到模型在真实环境中的鲁棒性。

Know-how：落到实现 / 训练系统

- 按 domain/language/style 做分桶保留，而非一个全局阈值。
- 用下游/held-out domain loss 验证过滤，而非只看 classifier AUROC。
- 保留 raw pool 与 clean pool，支持不同阶段重新采样。

高频追问

1. quality classifier 会有哪些 bias？

常见失分点

- 把质量分数当真值，不做模型闭环。

2. 极端高质量过滤为什么可能伤聊天能力？
3. 怎样量化 diversity？

一句话记忆 数据过滤不是“把坏的都删掉”，而是找到最有训练价值的分布。

题 17 预训练数据 Mixture 到底怎么调？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

数据配比是当代预训练的核心算法问题，决定不同能力的边际收益。

3 分钟主答

设 domain i 的采样概率为 p_i ，目标不是给出固定百分比，而是让有限 token budget 在多个能力/验证目标上获得最大边际收益。实际可通过 proxy models、不同 mixture 的 controlled experiments、domain-conditioned validation loss 与 scaling trend 来估计。

配比还受数据可用量、重复次数、阶段目标和 transfer effect 影响：同一 domain 在早期通用阶段与后期 reasoning/code 阶段的最优权重可能完全不同。

$$\min_p \sum_j w_j L_j(p), \sum_i p_i = 1$$

Know-how：落到实现 / 训练系统

- 先固定总 token 与模型大小，只改变 mixture 做正交对照。
- 记录每 domain 实际 consumed tokens，防止数据 loader 与重复策略偏离目标比例。
- 对稀缺高价值数据设置 epoch/repetition 上限并观察 overfit。

深挖关键点 面试若能主动提出 proxy model + mixture ablation + domain-conditioned loss，通常比背任何比例更有价值。

高频追问

1. 怎样判断数学数据该从 5% 加到 10%？
2. 不同 domain 的 loss 能直接比较吗？
3. mixture 与 curriculum 有什么区别？

常见失分点

- 报一组“经验百分比”却没有实验方法。
- 忽略不同 tokenizer 对 domain token 数的影响。

一句话记忆 配比不是 recipe，而是“每增加一个 token，应放到哪个 domain”的边际收益优化。

题 18 什么是 Benchmark Contamination? 如何系统检测?

B 类等价追问题

难度 难

频率 ★★★★★

面试官为什么问

考察评测可信度、数据治理与泄漏边界。

3 分钟主答

污染不只指 benchmark 问题原文进入训练集，还包括答案、解析、改写题、镜像页面和语义等价版本。Exact contamination 可用 n-gram/substrings; near duplicate 可用 MinHash/LSH; semantic contamination 可用 embedding retrieval 与人工复核。

检测必须在数据版本层面可重现，并区分“题目本身”“答案/解析”“同源模板”。否则 benchmark 提升可能只是记忆而非泛化。

Know-how: 落到实现 / 训练系统

- 建立 benchmark canonicalization: 去格式、标准化数字/空白后再匹配。
- 对候选污染输出 source URL、时间戳、相似片段供审计。
- 报告 contamination rate, 并在 clean subset 上重算评测。

高频追问

1. 只匹配 exact string 会漏掉什么?
2. 代码 benchmark 怎样做 contamination?
3. 训练前和训练后分别能做什么?

常见失分点

- 只删除测试题文件名对应页面。
- 把 contamination 当成单次脚本而没有版本审计。

一句话记忆 可信评测的前提是：不仅“题没见过”，连答案、解析和近似改写也要被审计。

题 19 Packing 与 Padding 有什么区别? 为什么 Packing 不是简单拼接?

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

公开面经高频题，能深入到 attention mask、loss weighting 和数据效率。

3 分钟主答

Padding 将不同长度样本补到统一长度，计算大量无效 PAD; Packing 把多个短文档装入同一固定长度 sequence，提高 token utilization。

但直接 concatenate 会让后一个文档看到前一个无关文档。是否允许跨文档 attention 取决于训练设计；若不允许，需要 document-aware mask / position reset。还要处理 EOS、loss mask、样本权重和边界位置。

Know-how: 落到实现 / 训练系统

- 统计 `useful_tokens / total_tokens`，而不是只看 `samples/sec`。
- 验证 `packed sequence` 的 `attention mask` 与 `label shift`，做小例子单元测试。
- 分布式 `dataloader` 中保证不同 `rank` 的 `token` 数接近，避免 `padding/packing` 造成 `straggler`。

深挖关键点 面试现场最好画出 `[A A EOS][B B EOS]` 并解释每一块 `attention mask`。

高频追问

1. 跨文档 `attention` 一定是错的吗？
2. `position id` 要不要 `reset`？
3. `packing` 会怎样改变 `sample weighting`？

常见失分点

- 把 `packing` 说成 `concatenate` 后加 `EOS` 就结束。
- 只看吞吐提升，不验证 `loss` 等价性。

一句话记忆 `Packing` 优化的是 `token utilization`；正确性取决于边界、`mask`、`position` 与 `loss` 四件事。

延伸阅读：R14

题 20 Synthetic Data 在预训练阶段的价值与风险是什么？

B 类等价追问问题

难度 中

频率 ★★★★★

面试官为什么问

合成数据已进入主流模型训练，需要理解信息增益而非“越多越好”。

3 分钟主答

`Synthetic data` 可以补足数学、代码、推理、稀缺语言等高价值域，控制难度和格式，并通过程序/规则做 `correctness filtering`。Qwen3 官方资料也描述了利用模型生成数学和代码数据。

风险包括 `teacher bias`、错误事实放大、风格同质化、真实分布覆盖下降以及递归训练导致的信息退化。核心指标应是 `information gain per token`，而不是合成 `token` 数。

Know-how: 落到实现 / 训练系统

- 给生成器、过滤器与最终训练模型分离版本，避免循环自证。
- 使用可验证任务优先：代码执行、数学答案、结构约束。
- 混入真实数据并做 `diversity/novelty` 监控。

高频追问

1. 如何防止 `teacher` 的错误被学生放大？
2. 为什么 `rejection sampling` 仍可能降低多样性？
3. 怎样衡量 `synthetic data` 的边际价值？

常见失分点

- 把合成数据等同于 `SFT` 数据。
- 只看过滤通过率，不测模型增益。

一句话记忆 合成数据真正要最大化的是“每个 token 带来的新、对、难的信息”。

延伸阅读：R11 / R12

第 3 章 · 题 21-30

目标函数、优化器与训练配方

从 next-token prediction 到 AdamW、Warmup、Batch Token 与阶段化训练。

本章面试策略

不要把数据与训练配方当固定 cookbook；面试官更关心你如何用实验找到合适方案。

题 21 Causal LM Loss 是什么？为什么 Next-Token Prediction 能学出复杂能力？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

从目标函数追到表示学习，是预训练算法岗的基本理论表达。

3 分钟主答

标准 causal LM 最大化每个 token 在历史条件下的似然，等价于最小化 cross-entropy。训练信号覆盖几乎每个位置，因此海量原始文本都能直接成为监督。

为了降低条件预测误差，模型必须压缩语法、语义、事实关系、代码结构和隐含状态等统计规律。所谓 reasoning 能力不能简单解释为“loss 神奇地产生了推理”，更准确是规模、数据与训练使模型学到支持多步预测的内部计算结构。

$$L = -\sum_t \log p_{\theta}(x_t \mid x_{<t})$$

Know-how：落到实现 / 训练系统

- 区分 token-averaged loss 与 sequence/sample weighting。
- 注意 padding、packing、特殊 token 是否参与 loss。
- 按 domain 分解 validation cross-entropy，避免总体 loss 掩盖能力。

高频追问

1. 为什么 perplexity=exp(loss)?
2. next-token loss 与 sequence-level quality 有何差异？
3. 如何给不同 token 加权？

常见失分点

- 用“涌现”替代机制解释。

一句话记忆 Next-token objective 的强大之处在于把几乎所有文本位置都变成训练信号。

题 22 为什么训练能并行预测所有 Token，而推理必须逐 Token 生成？

B 类等价追问题

难度 易

频率 ★★★★★☆

面试官为什么问

考察 teacher forcing、causal mask 与 prefill/decode 的区别。

3 分钟主答

训练时完整 ground-truth sequence 已知，所有位置的 Q/K/V 可一次计算；causal mask 只阻止位置 t 访问未来 token，所以各位置 loss 可以并行。

生成时 x_{t+1} 依赖刚生成的 x_t ，未来 token 不存在，形成串行 autoregressive dependency。工程上 prefill 仍可并行处理已有 prompt，而 decode 阶段每步通常只新增一个 token，并复用 KV Cache。

Know-how：落到实现 / 训练系统

- 明确区分 training、prefill、decode 三种 kernel/瓶颈。
- 训练通常 compute-bound，decode 更容易 memory-bandwidth/KV-cache bound。
- 吞吐分析用 tokens/s 与 batch/sequence 共同说明。

高频追问

1. Speculative decoding 如何缓解串行性？
2. 为什么 decode batch 变大能提高利用率？
3. KV Cache 在训练为什么通常不需要？

常见失分点

- 说“训练不是自回归”。训练目标仍是自回归，只是 teacher forcing 并行计算。

一句话记忆 训练的“并行”来自已知答案；推理的“串行”来自答案本身尚不存在。

题 23 AdamW 为什么长期是 LLM 预训练默认优化器？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

能从优化、自适应状态与显存代价继续追问。

3 分钟主答

Adam 维护一阶矩 m 与二阶矩 v ，对不同参数维度做自适应缩放；AdamW 将 weight decay 与梯度更新解耦，使 L2-like regularization 不再被 Adam 的 adaptive denominator 扭曲。

它的主要工程代价是 optimizer states： m 、 v 通常使用 FP32，加上 master weights/gradients 后每参数字节数很高，因此 ZeRO/FSDP/Distributed Optimizer 对大模型训练至关重要。

Know-how：落到实现 / 训练系统

- 回答公式后立刻手算 bytes/param，体现训练系统意识。
- 监控 update norm / parameter norm，而不只看 gradient norm。
- 大规模训练中 optimizer state checkpoint 也会成为 IO 瓶颈。

高频追问

1. AdamW 与 Adam+L2 有什么区别？
2. β_1/β_2 改变什么时间尺度？

常见失分点

- 只说“收敛快”。

3. 为什么 optimizer state 要用更高精度？

- 忘记 bias correction 或把 weight decay 写进梯度。

一句话记忆 AdamW 的价值是自适应更新；它的代价是昂贵的状态内存与 checkpoint。

题 24 为什么大模型训练需要 Learning-Rate Warmup?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察训练初期尺度、优化器状态和稳定性。

3 分钟主答

训练刚开始时参数与 activation 统计尚未适应数据，Adam 的 moment estimates 也处于建立阶段。直接使用 peak LR 容易产生过大的 early update，尤其深层/低精度模型更敏感。

Warmup 让学习率从小值渐升到目标峰值，为模型与 optimizer states 提供过渡。但如果模型因为初始化、norm 或 attention logits 本身不稳定，单纯拉长 warmup 只是掩盖根因。

Know-how: 落到实现 / 训练系统

- 同时看 loss、grad norm、update norm、activation RMS 的 warmup 曲线。
- 用 tokens 而不是 steps 定义 schedule 时更容易跨 global batch 比较。
- continued pretraining 是否 warmup 要看 distribution shift 和 optimizer state 是否继承。

高频追问

1. Warmup 多长合适？
2. Batch size 变大时 warmup 是否要改？
3. 从 checkpoint resume 需要重新 warmup 吗？

常见失分点

- 把 warmup 当成解决一切 NaN 的工具。

一句话记忆 Warmup 是控制“训练最脆弱阶段的更新幅度”，不是架构稳定性的替代品。

题 25 Cosine Decay 为什么常见？Continued Pretraining 如何重设 LR?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察 schedule 的机制与阶段切换。

3 分钟主答

Cosine decay 从 peak LR 平滑降到较小值，前期维持较强探索，后期逐渐减小更新幅度；它的优势更多是稳定、平滑和工程经验成熟，而非唯一理论最优。

Continued pretraining 若发生明显 domain/context 分布变化，常需要新的 warmup/较低 peak LR；若完全延续原 optimizer states 和同分布数据，则重启大 LR 可能破坏已有能力。决策必须基于 phase transition 的强度。

Know-how: 落到实现 / 训练系统

- 用 consumed tokens 记录每个 phase 的 LR，保证可重现。
- 阶段切换前后对通用域和目标域都做 held-out loss，观察遗忘。
- 小规模 sweep peak LR $\times$ warmup tokens。

高频追问

1. Cosine 与 linear decay 有什么实际差异？
2. 为什么 CTP 常用较小 LR？
3. optimizer states 重置会产生什么影响？

常见失分点

- 把 continued pretraining 当成“接着跑原 schedule”或“永远重置 optimizer”的固定答案。

一句话记忆 阶段化训练时，LR schedule 应跟着数据分布与 optimizer state 一起设计。

题 26 Batch Size 增大后，优化和系统分别发生什么？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察 gradient noise、吞吐与训练配方之间的耦合。

3 分钟主答

Batch 越大，随机梯度方差通常下降，单次 update 的方向更稳定；系统上则有更多 work 可摊薄通信和 kernel launch，提高设备利用率。但当 batch 超过某个有效区间，继续放大带来的优化收益递减，甚至需要更多 tokens 才达到同样泛化。

因此 batch 不能只按显存“塞满”。要联合调整 learning rate、warmup、tokens/update，并区分样本 batch 与 token batch。

Know-how: 落到实现 / 训练系统

- 记录 global batch tokens，而不是只报 samples。
- 用不同 batch 做等 token-budget 对照，避免把“step 更少”误当训练更快。
- 观察 gradient noise scale 或简单用 validation loss/token 判断临界区间。

高频追问

1. Large-batch 为什么常配更大 LR？

常见失分点

- 只说“大 batch 更稳定、更快”。

2. Batch 翻倍、grad accumulation 不变会影响什么？
3. 为什么 variable-length packing 下 samples/ batch 不可靠？

- 忽略 batch 增大改变优化轨迹。

一句话记忆 Batch size 同时是优化超参和系统超参；正确单位通常是 tokens/update。

题 27 Global Batch、Micro Batch、Gradient Accumulation 如何换算？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

这是分布式训练中最常见的口算题，直接检验是否真正跑过训练。

3 分钟主答

在标准数据并行语境下， $\text{global batch} = \text{micro batch per DP rank} \times \text{data-parallel degree} \times \text{gradient-accumulation steps}$ 。若每条样本固定长度 S ，则每次参数更新大约消费 $\text{global batch} \times S$ 个 token。

TP、PP 通常切同一个模型副本，不应再乘进 global batch。Pipeline 的 microbatch 概念还与 schedule/bubble 相关，面试时应先声明框架定义。

$$B_{\text{global}} = B_{\text{micro}} \times DP \times N_{\text{accum}}; \text{tokens/update} \approx B_{\text{global}} \times S$$

Know-how：落到实现 / 训练系统

- 从训练配置反推 consumed tokens，检查日志是否一致。
- batch schedule 变化时按 token 重新对齐 LR/warmup。
- variable-length packing 要用实际 non-padding tokens 统计。

深挖关键点 这是面试里最容易用一个数字暴露“只看过配置、没理解并行语义”的题。

高频追问

1. 为什么 TP 不乘进 global batch？
2. PP 的 microbatches 和 grad accumulation 是一回事吗？
3. sequence packing 后怎样计算 token batch？

常见失分点

- 把总 GPU 数直接乘进 batch。
- 不声明 micro-batch 是 per-rank 还是全局。

一句话记忆 先找 DP replica 数，再算 batch；模型并行 GPU 不是独立数据副本。

题 28 Weight Decay 为什么通常不施加到所有参数？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察正则化的作用对象以及训练 recipe 的非教条性。

3 分钟主答

Weight decay 对大矩阵权重提供尺度约束，但 bias、Norm scale 等参数的语义与普通线性权重不同，很多 recipe 会将其排除。原因不是“这些参数永远不能 decay”，而是 decay 对它们的 inductive bias 和收益常较弱。

Embedding 是否 decay、Norm 是否完全排除都没有统一定理，应以模型配方和 ablation 为准。

Know-how：落到实现 / 训练系统

- 构造 optimizer parameter groups 时按名字/维度明确记录。
- checkpoint resume 时验证 param-group 配置没有变化。
- 做小规模 ablation 看 loss 与参数 norm。

高频追问

1. Embedding 应不应该 decay？
2. 为什么 AdamW 是 decoupled decay？
3. weight decay 与 L2 regularization 在 Adam 下为何不同？

常见失分点

- 把“bias/norm 不 decay”当成数学必然。

一句话记忆 Decay 策略的核心是参数类型的尺度先验，而不是固定黑名单。

题 29 为什么 Label Smoothing 不是 LLM 预训练的默认配置？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

检验概率建模目标与分类技巧的差别。

3 分钟主答

语言建模希望拟合真实条件 token 分布；label smoothing 人为把 one-hot target 与均匀分布混合，会改变最大似然目标并降低模型对正确 token 的置信度。

在大词表生成任务中，目标本身已经具有高不确定性，而且 calibration/regularization 可通过数据规模、weight decay 等其他手段实现，因此 label smoothing 不像图像分类那样普遍。

Know-how：落到实现 / 训练系统

- 若使用 smoothing，必须明确对特殊 token、padding 和词表大小的处理。
- 比较 NLL、calibration 与生成质量，而不只看 top-1 accuracy。
- 不要把 SFT 的 soft targets 与预训练 label smoothing 混为一谈。

高频追问

1. Label smoothing 为什么可能改善 calibration？
2. 知识蒸馏 soft labels 与 smoothing 有什么差别？
3. 词表越大时均匀项有什么含义？

常见失分点

- 直接说“LLM 不需要正则化”。

一句话记忆 语言建模优化的是概率分布本身，因此随意修改 target distribution 要格外谨慎。

题 30 Multi-Token Prediction (MTP) 为什么重新受到重视？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察新一代预训练目标与训练效率。

3 分钟主答

标准 CLM 每个位置直接预测下一个 token；MTP 增加若干预测头/模块，让当前表示同时承担未来多个 token 的预测信号。这样可提高单位上下文位置的监督密度，并推动表示学习更具前瞻性。

DeepSeek-V3 将 MTP 纳入预训练设计。它带来额外参数/FLOPs和 loss weighting 问题；推理时这些辅助模块是否保留、如何用于 speculative decoding，则是另一层系统设计。

Know-how：落到实现 / 训练系统

- 分别记录主 LM loss 与各 MTP depth loss。
- 做等 FLOPs 或等 wall-clock 对照，避免仅因多算导致提升。
- 检查辅助目标是否损害主 next-token calibration。

高频追问

1. MTP 与一次预测多个独立 token 有何不同？
2. 辅助头共享哪些参数？
3. 为什么它可能帮助 speculative decoding？

常见失分点

- 把 MTP 说成“模型一次生成多个 token”而忽略训练目标。

一句话记忆 MTP 的核心是增加未来监督信号；它是否带来端到端收益必须按等成本比较。

延伸阅读：R10

第 4 章 · 题 31-40

Scaling Law 与算力预算

把参数规模、训练 Token、FLOPs、GPU-hours 与推理成本统一起来。

本章面试策略

不要把数据与训练配方当固定 cookbook；面试官更关心你如何用实验找到合适方案。

题 31 什么是 Scaling Law？预训练团队为什么需要它？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

从“训练一个模型”上升到“规划一族模型”的核心研究能力。

3 分钟主答

Scaling law 描述 validation loss 与模型参数 N 、数据量 D 、训练 compute C 之间近似幂律关系。它让团队可以用较便宜的 proxy runs 拟合趋势，再预测大规模训练的 loss、最优 token/parameter 配比与预算风险。

它不是物理定律：模型架构、tokenizer、数据 mixture、训练 recipe 改变后，拟合参数可能变化，因此必须在目标设置附近重新校准。

$$L(N,D) \approx L_{\infty} + aN^{-\alpha} + bD^{-\beta}$$

Know-how：落到实现 / 训练系统

- 设计多个 $N \times D \times \text{recipe}$ 的正交小实验，而不是只做一条 size curve。
- 使用 held-out loss 且保证数据 pipeline 一致。
- 报告拟合区间与外推距离，避免过度 extrapolation。

深挖关键点 优秀回答要强调“可用于预算决策，但必须有误差条和适用域”。

高频追问

1. Scaling law 能预测 benchmark accuracy 吗？
2. 架构改变后还能复用曲线吗？
3. 如何判断 proxy model 足够代表大模型？

常见失分点

- 把幂律当成任何规模都精确成立。
- 只拟合两个点就外推百倍规模。

一句话记忆 Scaling law 是用小规模实验降低大规模决策不确定性的工具。

延伸阅读：R5

题 32 Kaplan Scaling 与 Chinchilla 的关键差异是什么？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

经典题，考察 compute-optimal 训练观念的演变。

3 分钟主答

早期 scaling 结果促使行业偏向更大参数模型；Chinchilla 系统研究固定 compute 下模型大小与训练 token 的联合扩展，指出许多大模型相对 undertrained。

面试不应死背“参数和 token 同比例”这句话，而应理解：固定训练 FLOPs 时存在 N 与 D 的最优分配；如果部署成本也纳入目标，最优解还会改变。

Know-how: 落到实现 / 训练系统

- 画出固定 compute 约束下 N 与 D 的 trade-off。
- 说明 tokenizer 变化会改变 token 这一计量单位。
- 将训练最优与生命周期成本最优区分。

高频追问

1. Chinchilla 为什么影响模型尺寸选择？
2. 为什么今天很多小模型训练远超 Chinchilla token？
3. MoE 如何放进 compute-optimal 比较？

常见失分点

- 把 Chinchilla 结论当成固定 20 tokens/param 的永久规则。

一句话记忆 Chinchilla 教会我们：固定算力下，模型容量和数据量必须一起优化。

延伸阅读：R5

题 33 为什么今天很多模型会“过训练”更小的参数规模？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察训练成本与长期推理成本的统一经济学。

3 分钟主答

若只优化一次性 pretraining FLOPs, compute-optimal 点是一种答案；但现实模型会被推理数十亿次。更小模型即便训练更多 token, 只要推理 FLOPs、显存与 latency 显著更低, 生命周期总成本可能更优。

因此现代模型设计常做 inference-aware scaling: 训练阶段允许更多 compute, 以换取部署阶段更小 active parameters、更低 KV/内存成本。

Know-how: 落到实现 / 训练系统

- 把目标写成 training cost + expected lifetime inference cost。
- 分别测模型能力随额外 token 的边际收益和 serving cost。
- 考虑蒸馏、MoE 等替代策略。

高频追问

1. 什么情况下应选更大的 undertrained 模型？
2. 推理请求量未知怎么办？
3. 模型过训练是否意味着数据重复风险？

常见失分点

- 把“过训练”理解成 train loss 过拟合。

一句话记忆

所谓 overtraining smaller model，通常是相对 compute-optimal 训练预算而言，不等于统计过拟合。

延伸阅读：R5

题 34 为什么训练 FLOPs 常粗略写成 6ND?

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

这是预算、GPU-hours 和 scaling law 的基础手算。

3 分钟主答

对于参数量 N 、训练 token D 的 dense Transformer，单 token forward 中每个参数参与乘加可粗略记为约 $2N$ FLOPs；backward 对 activation/weight gradient 的代价约为 forward 的两倍，因此训练总量常粗估为 $6ND$ 。

这只是工程近似。Attention 的 S^2 项、embedding、MoE 稀疏激活、activation recompute、optimizer/kernel overhead 都会造成偏差。

$$C_{\text{train}} \approx 6ND$$

Know-how：落到实现 / 训练系统

- 预算先用 $6ND$ 给量级，再用 profiler/框架 FLOP counter 校正。
- GPU-hours 换算必须乘实际 MFU，而不是 peak FLOPs。
- MoE 要用 activated parameters/FLOPs，而不是 total params 直接代入。

深挖关键点 面试手算要先给量级，再列出会偏离该近似的项。

高频追问

1. 为什么 backward 约是 forward 两倍？
2. checkpointing 会不会改变 $6ND$ ？
3. 怎么从 $6ND$ 估训练天数？

常见失分点

- 把 $6ND$ 当精确公式。
- 直接用 GPU 峰值算 wall-clock，不乘利用率。

一句话记忆 $6ND$ 是预算尺，不是 profiler。

延伸阅读：R5

题 35 给定固定 GPU-hours，如何决定模型大小和 Token 数？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

最接近真实预训练立项的系统设计题。

3 分钟主答

先把 GPU 数、时长、峰值算力与预期 MFU 转成可用训练 FLOPs，再利用 scaling-law proxy runs 在候选 N/D 上寻找低 loss 区域。之后加入硬件约束：显存能否 fit、TP/PP 并行效率、数据供应和目标上下文。

最终还要加入部署目标。一个数学上 compute-optimal 的超宽模型，可能因 TP 通信或推理成本而不是系统最优。

Know-how：落到实现 / 训练系统

- 先做粗预算，再做 100M/1B 级 proxy grid。
- 对每个候选记录预计训练天数、MFU、内存余量和推理成本。
- 保留 5-10% 预算用于 pilot、失败恢复和关键 ablation。

深挖关键点 先算得起，再算最优，最后算能不能稳定跑完。

高频追问

1. 如果数据只有 2T 高质量 token 怎么办？
2. MFU 估错 10% 会怎样影响计划？
3. 为什么系统最优可能偏离 scaling-law 最优？

常见失分点

- 拿到预算直接选行业常见模型大小。

一句话记忆 立项不是“选参数”，而是把算力、数据、架构和部署约束联合求解。

延伸阅读：R5 / R9

题 36 为什么 1B 模型的最优超参不能原样搬到 100B？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察 scale-dependent optimization 与 proxy experiment 的边界。

3 分钟主答

模型规模变化会改变梯度统计、深度、activation scale、critical batch size、最优 LR、通信比例和数值稳定性；系统并行方式也会改变有效 batch 与 kernel shape。

因此小模型的价值是拟合趋势和筛掉明显坏方案，而不是给出大模型全部超参的精确值。需要跨多个规模建立 scaling trend，并保留大规模 pilot。

Know-how：落到实现 / 训练系统

- 至少用 2-3 个规模观测 LR/batch/稳定性趋势。
- 保持数据/tokenizer/训练目标一致，避免混杂因素。

- 对大模型先跑短 horizon stability test。

高频追问

1. 哪些超参最容易随规模变化？
2. Initialization 是否也需要 scale？
3. MoE router 超参能否从 dense proxy 推？

常见失分点

- 把 proxy model 当缩小版真模型。

一句话记忆 小模型提供方向，大模型仍需要规模特定的稳定性与系统校准。

题 37 如何用小模型实验预测大模型最终 Loss？

B 类等价追问问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察实验设计、拟合与外推可信度。

3 分钟主答

构造多个模型规模 N 、token 数 D 的训练点，统一 tokenizer、数据 mixture 和 recipe，得到 held-out loss；用 power-law 类函数拟合并估计参数置信区间，再在未参与拟合的中等规模点做 holdout 验证。

真正关键的是误差管理：外推距离越远、架构差异越大，预测风险越高。应同时给 point estimate、confidence interval 和对 mixture/recipe 改动的敏感性。

Know-how：落到实现 / 训练系统

- 预先定义训练预算网格，避免只保留“成功点”。
- 对拟合残差做检查，识别 regime change。
- 用中型模型作为 bridge point 验证 extrapolation。

高频追问

1. 为什么不能只用最终 benchmark 拟合？
2. 数据质量变化会怎样移动 curve？
3. 怎样判断出现新的 scaling regime？

常见失分点

- 只报 R^2 很高，不看外推误差。

一句话记忆 Scaling 预测的专业度主要体现在“知道自己可能错多少”。

延伸阅读：R5

题 38 Loss 平滑 Scaling，为什么能力可能出现阈值或跳变？

B 类等价追问题

难度 难

频率 ★★☆☆☆

面试官为什么问

考察连续优化指标与离散 benchmark 的差异。

3 分钟主答

Cross-entropy 是对海量 token 的连续平均指标，通常随规模平滑下降；而 benchmark accuracy 常由离散决策、复杂链路或 exact match 组成，小幅概率变化可能让输出跨过正确/错误边界。

因此“能力涌现”中一部分现象可能来自度量方式的非线性。不能因为 loss 平滑就断言所有能力平滑，也不能因 benchmark 跳变就断言模型内部突然产生全新机制。

Know-how：落到实现 / 训练系统

- 同时画 continuous metric（NLL、pass@k 概率）与 discrete accuracy。
- 对任务难度分桶，寻找阈值是否来自测量。
- 用多个规模与随机种子排除噪声。

高频追问

1. 什么是 metric-induced emergence？
2. 为什么 exact-match 尤其容易产生阈值？
3. 怎样构造更平滑的能力指标？

常见失分点

- 把 benchmark 跳变直接神秘化为“突然智能”。

一句话记忆 先区分模型变化与测量函数变化，再讨论涌现。

题 39 同样 FLOPs 下，Dense 与 MoE 怎么公平比较？

B 类等价追问题

难度 难

频率 ★★★★★

面试官为什么问

避免用 total parameters 制造错误比较。

3 分钟主答

MoE 的 total parameters 很大，但每个 token 只激活少数 experts。因此公平比较至少要对齐 training FLOPs/token、训练 tokens、wall-clock 或总预算，同时报告 total params 与 activated params。

还必须计入 routing、All-to-All、load imbalance 和 kernel 效率。算法 FLOPs 相近不意味着集群成本相近。DeepSeek-V3 例如公开为 671B total、约 37B activated/token。

Know-how：落到实现 / 训练系统

- 制作比较表：total/active params、FLOPs/token、tokens、MFU、network time、memory。

- 用实际 accelerator-hours 做第二组成本指标。
- 统一数据和 evaluation。

高频追问

1. MoE 的 embedding/attention 是不是也稀疏？
2. activated params 能完全代表 FLOPs 吗？
3. 为什么 MoE wall-clock 可能比 dense 更差？

常见失分点

- 拿 671B 与同 FLOPs 的 671B dense 直接比较。

一句话记忆 MoE 比较必须看 active compute 与通信，而不是只看总参数。

延伸阅读：R10

题 40 Scaling Law 能否用于 Data Mixture 优化？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

连接 scaling 与数据配方，是更偏研究型的开放题。

3 分钟主答

可以把验证 loss 看作 N 、各域 token D_i 或采样比例 p_i 的函数，通过多组小模型 mixture experiments 拟合不同 domain 的边际收益与 transfer effect。

难点在于不同域之间不是独立可加：代码可能促进 reasoning，数学数据可能影响通用语言；高重复域还会出现收益饱和。因此需要多维实验和受控简化，而不是简单为每个域拟合一条曲线。

Know-how：落到实现 / 训练系统

- 先选择 3-5 个高影响域做 mixture grid，避免维度爆炸。
- 按 target capability 构造加权 validation objective。
- 定期在训练阶段重新估计边际收益，支持 curriculum。

高频追问

1. 如何处理 domain 间 transfer？
2. 数据不足导致重复时 scaling 怎么变？
3. mixture 优化目标是 loss 还是 benchmark？

常见失分点

- 假设每个域完全独立。

一句话记忆 Data scaling 的终极问题不是“要多少数据”，而是“下一批 token 应来自哪里”。

延伸阅读：R5

第 5 章 · 题 41-50

分布式训练

DP / TP / PP / SP / CP / EP / FSDP：切什么、怎么通信、为什么这样组合。

本章面试策略

先把核心公式 / Shape 写出来，再解释设计动机；最后一定落到显存、通信、吞吐或数据分布。

题 41 DDP 到底做了什么？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

所有并行训练问题的基线，必须清楚 replica、gradient 与 collective。

3 分钟主答

DDP 在每个 data-parallel rank 保存完整模型，处理不同 micro-batch，独立 forward/backward；梯度生成后通过 AllReduce（或等价 reduce-scatter/all-gather 组合）求和/平均，使所有 replica 得到一致梯度并执行相同 optimizer update。

DDP 降低的是“每步耗时通过并行数据”，并不降低每个 rank 的模型/optimizer state 显存，因此超大模型需要 FSDP/ZeRO 或模型并行。

Know-how：落到实现 / 训练系统

- 画出 4 rank 的 gradient bucket 与 overlap。
- 检查 effective global batch 与 gradient averaging 语义。
- 通信瓶颈分析看 bucket size、网络拓扑和 compute/comm overlap。

高频追问

1. AllReduce 在 backward 何时触发？
2. 为什么 DDP 不省参数显存？
3. 如果一个 rank 梯度 NaN 会怎样？

常见失分点

- 说 DDP 会把模型参数自动切到多卡。

一句话记忆 DDP 复制模型、切数据、同步梯度；它不是模型分片。

延伸阅读：R9 / R13

题 42 Tensor Parallel 如何切 Transformer?

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

Megatron 核心题，检验矩阵分片与通信理解。

3 分钟主答

对线性层 $Y=XW$ ，可按 W 的列切分让各 rank 产生部分输出（column parallel），或按行切分让各 rank 计算部分和并做 reduce（row parallel）。Megatron 将这种互补切法组合到 Attention QKV/O projection 与 MLP 中。

TP 在层内发生，几乎每层都有 collective，因此通信频率高；好处是单层参数/activation 可分布到多个 GPU，并保持大 GEMM 并行。

Know-how：落到实现 / 训练系统

- 白板写出矩阵 shape 和每 rank shape，再标 AllReduce/AllGather。
- head 数、hidden、FFN dim 要能被 TP degree 整除。

- profile 时看通信是否与 GEMM overlap，以及跨节点链路。

深挖关键点 能画清两层 MLP 的 column-parallel → activation → row-parallel，基本就掌握了 TP 核心。

高频追问

1. Column-parallel 为什么第一层可不立即 all-gather?
2. Attention heads 怎么分?
3. TP=8 为什么可能比 TP=4 更慢?

常见失分点

- 只说“把模型横着切”。
- 不会指出哪一步需要 collective。

一句话记忆 TP 的答案必须包含：切哪个矩阵维度、每卡算什么、在哪里通信。

延伸阅读：R8 / R9 / R13

题 43 为什么 TP 通常限制在 NVLink/NVSwitch 高速域?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察 topology-aware parallelism。

3 分钟主答

TP 属于层内模型并行，collective 出现频率与层数同量级；若跨低带宽/高时延网络，通信会直接进入每层 critical path。

因此常见策略是把 TP 放在单节点或高速岛内，把 DP/PP 用于更远拓扑。但这是经验规则：具体还要看节点 GPU 数、网络、模型 shape 和 collective 实现。

Know-how：落到实现 / 训练系统

- 先获取实际 topology 和 NCCL bandwidth，而非按“节点内/节点外”标签推理。
- 对不同 TP degree 做 end-to-end tokens/s sweep。
- 确保进程映射让 TP group 尽量拓扑局部。

高频追问

1. 为什么 PP 更适合跨节点?
2. InfiniBand 足够快时能否跨节点 TP?
3. TP 对小模型为什么可能得不偿失?

常见失分点

- 把 NVLink 当成使用 TP 的必要条件。

一句话记忆 高频通信的并行轴应优先映射到最快、最局部的互联。

题 44 Pipeline Parallel 为什么有 Bubble? 1F1B 如何改善?

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

考察时间维度并行与 schedule。

3 分钟主答

PP 将不同层放到不同 stage。流水线开始时后续 stage 等待输入，结束时前面 stage 等待反向传播，形成 warmup/drain bubble；microbatch 数越少、stage 越多，空闲比例通常越高。

1F1B 在 warmup 后交替执行一个 forward 与一个 backward，使 activation 更快释放并提高 steady-state 利用率；interleaved/virtual pipeline 进一步细化 stage 以降低 bubble。

Know-how：落到实现 / 训练系统

- 画 Gantt chart 比文字更清楚。
- 监控每 stage step time，避免一个 stage 计算量更大形成额外 bubble。
- microbatch 数既影响 bubble，也影响 global batch 与 activation memory。

高频追问

1. GPipe 与 1F1B 差别？
2. 为什么 virtual pipeline 能减 bubble？
3. PP stage 怎么平衡层数？

常见失分点

- 只说“PP 会等待”，不指出 warmup/drain 与 stage imbalance。

一句话记忆 PP 的效率问题既有调度 bubble，也有 stage 负载不均。

题 45 为什么 1F1B 比 All-Forward-Then-Backward 更省 Activation Memory?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

把 pipeline schedule 与显存生命周期联系起来。

3 分钟主答

若先完成大量 microbatch forward 再 backward，每个 microbatch 的中间 activation 都必须存到对应 backward，峰值随 in-flight microbatches 增大。

1F1B 在稳态中尽早对较早 microbatch 做 backward，可释放其 activation，显著降低同时驻留的 activation 数量。代价是调度依赖更复杂。

Know-how：落到实现 / 训练系统

- 用 activation lifetime 图解释峰值，而非只背结论。
- 结合 activation checkpointing 看 compute-memory trade-off。
- 测 stage-specific peak memory，避免平均值掩盖 OOM stage。

高频追问

1. 1F1B 能把 activation 降到 $O(1)$ 吗？
2. 为什么 warmup 阶段仍会积累 activation？
3. interleaved schedule 对显存有什么影响？

常见失分点

- 认为只要 PP 就自动省 activation。

一句话记忆 显存取决于“有多少 microbatch 的 activation 同时活着”，schedule 决定其生命周期。

延伸阅读：R9

题 46 ZeRO-1、ZeRO-2、ZeRO-3 分别 Shard 什么？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

经典显存题，也是 FSDP/Distributed Optimizer 的基础。

3 分钟主答

ZeRO Stage 1 shard optimizer states; Stage 2 进一步 shard gradients; Stage 3 再 shard model parameters。目标是消除 data-parallel replicas 对 model states 的冗余。

分得越彻底，单卡常驻显存越低，但 forward/backward 需要更频繁地 all-gather 参数或 reduce-scatter 梯度，通信与调度复杂性上升。

Know-how：落到实现 / 训练系统

- 手算时分开 P（参数）、G（梯度）、O（optimizer）三类字节。
- 明确每一类是 replicated 还是 /DP。
- 评估 stage3 的 parameter prefetch / reshard 与网络带宽。

深挖关键点 面试手算建议画 P/G/O 三行表，再对 Stage 1/2/3 标“复制/分片”。

高频追问

1. ZeRO-2 为什么 forward 不需要 all-gather 参数？
2. Stage3 每层参数什么时候聚合？

常见失分点

- 只背 1/2/3 列表，算不出单卡显存。

3. ZeRO 与 TP 可以叠加吗？

一句话记忆 ZeRO 的本质是沿 DP 组消除 model-state replication，级别越高通信越多。

延伸阅读：R7 / R13

题 47 FSDP 与 ZeRO-3 有什么关系和区别？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

考察概念等价性与框架实现差异。

3 分钟主答

概念上两者都可做到参数、梯度、optimizer state 的全分片：计算某个模块前聚合所需参数，反向后对梯度 reduce-scatter，并可重新 shard 参数。

区别更多体现在框架 abstraction、sharding unit、prefetch/reshard 策略、混合精度与生态。不能只回答“一个 PyTorch、一个 DeepSpeed”，应先讲共同的 full-shard 原理。

Know-how：落到实现 / 训练系统

- 比较实际配置时列 sharding granularity、prefetch、CPU offload、checkpoint format。
- 关注模型 wrap policy 是否导致过小 all-gather。
- 测端到端通信与 peak memory。

高频追问

1. FSDP1/FSDP2 思路有什么变化？
2. 为什么 wrap granularity 很重要？
3. FSDP 能和 TP/PP 组合吗？

常见失分点

- 把产品名当算法定义。

一句话记忆 先说共同的 full sharding，再说实现 abstraction；这才是技术比较。

延伸阅读：R9 / R13

题 48 Sequence Parallel 与 Context Parallel 有什么区别？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

新一代长上下文训练的高频区分题。

3 分钟主答

Sequence Parallel (Megatron 语境) 通常把 LayerNorm、Dropout 等 TP 区域之外的 activation 沿 sequence 维切分, 减少冗余 activation; Attention 的完整 context 语义仍由 TP 相关通信保证。

Context Parallel 则把整个网络输入/activation 沿 sequence 维分到多个 rank, Attention 的 Q 需要访问全局 K/V, 因此必须进行 KV 交换/环式通信。CP 针对长 context 的 activation memory 更直接。

Know-how: 落到实现 / 训练系统

- 回答前声明框架语境, 不同库对“SP”命名可能不同。
- 画出每 rank 持有的 sequence slice。
- 指出 CP 的核心通信对象是 K/V, 而不是普通 DDP gradient。

深挖关键点 区分二者最有效的问题: 每个 rank 的 Attention 是否拥有完整 sequence? 如果没有, KV 怎么补齐?

高频追问

1. SP 为什么常与 TP 配对?
2. CP 与 Ring Attention 的关系?
3. CP degree 增大为什么可能通信受限?

常见失分点

- 把 SP 与 CP 都说成“切 sequence”。
- 不说明 Attention 如何看到全局 KV。

一句话记忆 SP 更像减少 TP 周边 activation 冗余; CP 是把长 context 本身跨卡分治。

延伸阅读: R9

题 49 Expert Parallel 为什么需要 All-to-All?

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

MoE 系统的核心通信题。

3 分钟主答

Router 对每个 token 选出的 expert 可能位于其他 rank。为了把 token hidden states 发送到对应 expert, 需要 token dispatch; expert MLP 完成后又要把结果返回原 token 顺序做 combine, 因此典型实现出现两次 All-to-All 类通信。

这使 MoE 从“稀疏计算省 FLOPs”转化为“通信与负载均衡挑战”。expert 越碎、每 expert token 越少, communication/computation ratio 越高。

Know-how: 落到实现 / 训练系统

- 画 router→dispatch→expert→combine 数据流。
- 监控 tokens/expert、A2A bytes、等待时间和 grouped GEMM utilization。

- 让 EP group 映射到高带宽拓扑并尝试 overlap。

高频追问

1. 如果 expert 在本 rank 为什么也要参与 collective?
2. capacity factor / token drop 怎么影响通信?
3. EP 与 DP group 如何组合?

常见失分点

- 只说 “MoE 要通信”，说不出发送的是什么 tensor。

一句话记忆 EP 的 All-to-All 本质是把 token 按路由结果重新排列到 expert 所在设备。

延伸阅读：R9 / R10

题 50 给你 1024 张 GPU，TP/PP/DP/CP/EP 怎么设计？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

综合考察显存约束、通信频率、拓扑和模型形态。

3 分钟主答

先用模型是否能 fit 决定最小模型并行：单层/大矩阵需要 TP；深度或总参数需要 PP；长序列 activation 需要 CP；MoE experts 需要 EP。满足这些约束后，剩余资源尽量给 DP，因为 DP 通常通信频率相对低且吞吐扩展更自然。

并行度不是整数分解游戏。要考虑 TP/EP 的交互、PP stage balance、网络拓扑与 microbatch。框架中某些轴可能共享/折叠，必须按实际并行 group 定义核对。

$$N_GPU \approx TP \times PP \times CP \times EP \times DP \text{ (具体轴关系以框架为准)}$$

Know-how：落到实现 / 训练系统

- 先写 GPU factorization，再画 node/rack 映射。
- 用 profiler 比较 compute、collective、bubble 三类时间。
- 从最小必要 TP/PP/CP/EP 出发，不盲目把 model parallel 开大。

深挖关键点 真正高分回答会先向面试官索要模型层数/hidden/context、节点拓扑与网络带宽，再给数字。

高频追问

1. 为什么先最大化 DP 常是好策略？
2. 如果 128K context，CP 应怎样介入？
3. MoE attention 与 expert 能否用不同 TP？

常见失分点

- 只给 TP=8、PP=16、DP=8 一组数字，不说明模型和拓扑。

- 忽略 microbatch 过小导致 PP/TP GEMM 利用率下降。

一句话记忆 大规模并行设计顺序：先满足 fit，再减少高频通信，最后最大化可扩展的数据并行。

延伸阅读：R9

第 6 章 · 题 51-60

显存、精度与训练性能

会手算显存，会解释 BF16/FP8，会区分 compute wall、memory wall 与 communication wall。

本章面试策略

先把核心公式 / Shape 写出来，再解释设计动机；最后一定落到显存、通信、吞吐或数据分布。

题 51 7B 模型 BF16 参数本身占多少显存？为什么训练远不止这些？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

最基础的训练显存口算，用来确认 bytes/param 思维。

3 分钟主答

BF16 每参数 2 bytes，因此 7B 权重约 14GB（十进制量级）。但训练还需要 gradient、optimizer moments、可能的 FP32 master weights，以及 activation、临时 workspace、通信 buffer 和 allocator fragmentation。

所以“模型权重能放下”不等于“能训练”。必须区分 model-state memory、activation memory 与 transient peak。

$$7 \times 10^9 \text{ params} \times 2 \text{ bytes} \approx 14 \text{ GB}$$

Know-how：落到实现 / 训练系统

- 先列持久状态再列动态状态，避免漏项。
- 显存预算保留 10%-20% 峰值余量，实际以 profiler 为准。
- 用 ZeRO/FSDP 后先分片 model states，再估 activation。

高频追问

1. 为什么 14GB 不代表 16GB 卡能训练 7B？
2. GiB 与 GB 差多少？
3. 推理时还要加什么主要显存？

常见失分点

- 只算 weights。
- 把 checkpoint 文件大小直接等同训练显存。

一句话记忆 显存手算第一步：参数量 $\times$ dtype；第二步才是真正的训练状态清单。

题 52 Adam 混合精度训练每参数到底多少 Bytes？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

高频手算题，能一路追到 ZeRO/FSDP。

3 分钟主答

必须先声明假设。一个常见示例：BF16 参数 2B + BF16 梯度 2B + FP32 master weight 4B + FP32 Adam m 4B + FP32 Adam v 4B = 16 bytes/param。不同实现可能没有独立 master weights，gradient dtype 也可能不同，所以数字不是固定答案。

70B 若按 16B/param，model states 就约 1.12TB，尚未计 activation 与 buffer；这说明大模型训练为什么必须分片。

$$\text{示例: } 2(P) + 2(G) + 4(\text{master}) + 4(m) + 4(v) = 16 \text{ bytes/param}$$

Know-how: 落到实现 / 训练系统

- 面试白板画 Param/Grad/Master/m/v 五行表。
- 应用 ZeRO 时逐行判断 replicated 还是 sharded。
- 核对具体框架 distributed optimizer 文档，不背一个“万能 16B”。

深挖关键点 这是 S 级必练题：要能在 2 分钟内从单卡状态算到 ZeRO-1/2/3。

高频追问

1. ZeRO-1 后每卡多少？
2. 如果 grad 是 FP32 呢？
3. BF16 optimizer states 可不可以？

常见失分点

- 不声明假设就报一个固定数字。
- 忘记 activation，或把 activation 平均按 param 算。

一句话记忆 显存题的高分关键不是数字，而是“假设透明 + 逐项核算 + 分片语义正确”。

延伸阅读：R9 / R15

题 53 Activation Memory 和 Parameter Memory 谁更大？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察动态显存随 batch/sequence/depth 的增长。

3 分钟主答

没有统一答案。Parameter states 主要随参数 N 增长；activation 则大体随 microbatch B 、sequence S 、hidden H 、layers L 增长，并受 Attention/MLP 中间张量与 checkpointing 强烈影响。

长上下文或大 microbatch 下 activation 可能成为主导；大参数、小 batch 时 optimizer/model states 更突出。面试应先问配置，再判断瓶颈。

Know-how: 落到实现 / 训练系统

- 用 framework memory snapshot 找峰值 tensor。
- 分别 sweep microbatch 与 sequence，看峰值斜率。
- 长上下文优先考虑 CP、checkpointing 与 memory-efficient attention。

高频追问

1. 为什么 FlashAttention 后 activation 仍可能很大？
2. PP 会如何改变 activation？
3. microbatch=1 还可能 activation OOM 吗？

常见失分点

- 给出绝对结论“optimizer 一定最大”。

一句话记忆 显存瓶颈是配置函数：参数规模决定底座， $B \times S \times H \times L$ 决定 activation 压力。

题 54 Activation Checkpointing 的原理与代价是什么？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

公开面经高频 OOM 题，核心是 compute-for-memory。

3 分钟主答

正常 backward 需要 forward 中间 activation。Checkpointing 只保留一部分边界 activation，反向时重新执行缺失的 forward 片段，从而显著降低峰值 activation memory。

代价是额外 FLOPs、kernel launch 与可能的通信重算。选择粒度过细会让 recomputation 成本很高；过粗又省不了足够显存。

Know-how：落到实现 / 训练系统

- 先找最大 activation 的模块，再选择 selective/full recompute。
- 同时测 peak memory、step time、MFU，不只看是否 OOM。
- 确保 dropout/RNG state 在重算时一致。

高频追问

1. Checkpointing 为什么会影响 RNG？
2. Selective recompute 怎么选？
3. 它能降低 optimizer memory 吗？

常见失分点

- 说成“把 activation 存 CPU”。那是 offload，不是 recompute。

一句话记忆 Checkpointing 用额外计算换 activation 显存；它不解决参数/optimizer state 复制。

延伸阅读：R13 / R9

题 55 为什么 BF16 通常比 FP16 更适合 LLM 预训练？

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

考察浮点格式与 overflow/underflow。

3 分钟主答

BF16 与 FP32 都有 8-bit exponent，因此动态范围接近 FP32；FP16 exponent 只有 5 bit，虽然 mantissa 更精细，但更容易在大范围梯度/activation 中 overflow。

LLM 训练通常更看重动态范围而非额外尾数精度，因此 BF16 更省去繁琐 loss scaling。但关键算子仍可能需要 FP32 accumulation。

Know-how：落到实现 / 训练系统

- 明确 storage dtype、compute dtype、accumulation dtype 是三件事。
- 监控 Inf/NaN 与 underflow，不以“BF16 不会炸”自满。
- Norm、softmax、reduction 关注高精度累积。

高频追问

1. FP16 为什么需要 loss scaling?
2. BF16 精度更低为什么还能训?
3. TF32 与 BF16 有什么不同?

常见失分点

- 说 BF16 “比 FP16 精度更高”。它是动态范围更大、尾数更少。

一句话记忆 BF16 的优势是 FP32 级 exponent range，用精度换更安全的动态范围。

延伸阅读：R16

题 56 FP8 Training 的核心难点是什么？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察低精度训练是否理解 scale 管理，而非只看位宽。

3 分钟主答

FP8 常使用不同 exponent/mantissa 配置，并通过动态 scaling 将 tensor 映射到可表示范围。难点在于不同层/不同时间的数值范围变化很大，单一 scale 容易 overflow 或丢失小值。

现代方案会用 per-tensor/per-channel/per-block scale、amax history、较高精度 accumulation，并让敏感算子保留 BF16/FP32。收益来自更高 Tensor Core 吞吐和更低带宽，但稳定性必须实证。

Know-how：落到实现 / 训练系统

- 记录 overflow rate、scale/amax 分布与 layer-wise error。
- 先在成熟 BF16 baseline 上做等 token 对照。
- 对 QK、Norm、loss 等敏感路径单独检查。

高频追问

1. E4M3 与 E5M2 怎样取舍?
2. 为什么 block scaling 更稳但有元数据开销?
3. FP8 会不会一定快 2 倍?

常见失分点

- 把 FP8 等同于“把 dtype 改成 float8”。

一句话记忆 低精度训练的核心不是位数，而是“如何持续把动态范围放进有限编码空间”。

题 57 什么是 Arithmetic Intensity? 为什么 LLM 工程必须懂?

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

把算法 FLOPs 与真实硬件瓶颈连接起来。

3 分钟主答

Arithmetic intensity = 执行 FLOPs / 从内存层级搬运的 bytes。若 AI 低，性能更受内存带宽限制；AI 高且矩阵足够大，才更可能接近 compute-bound。

这解释了为什么某个 FLOPs 更少的实现不一定更快：它可能产生更多 HBM traffic、小 GEMM 或同步。Roofline 思维比只看理论 FLOPs 更接近端到端训练。

$$AI = \text{FLOPs} / \text{bytes moved}$$

Know-how: 落到实现 / 训练系统

- profile kernel 的 achieved FLOPs、HBM bandwidth 与 occupancy。
- 通过 tiling/fusion/batching 提高数据复用。
- 将 kernel 优化最终映射到 step time / tokens/s。

高频追问

1. Decode 为什么常比 training 更 memory-bound?
2. 小 GEMM 为什么利用率低?
3. FlashAttention 怎样提高有效 AI?

常见失分点

- 把 AI 与 GPU utilization 混为一谈。

一句话记忆 FLOPs 告诉你算多少，Arithmetic Intensity 告诉你更可能卡在算力还是搬数据。

延伸阅读: R6

题 58 FlashAttention 为什么快? 它是不是近似 Attention?

A 类高频真题型

难度 中

频率 ★★★★★★

面试官为什么问

必考题，最常见错误是把 IO 优化说成稀疏/近似。

3 分钟主答

FlashAttention 计算的是 exact dense attention。它通过 tiling 把 Q/K/V 分块搬入片上 SRAM，使用 online softmax 在块间维护归一化统计，避免把完整 $S \times S$ score/probability matrix 反复写入 HBM。

因此核心收益来自降低 IO complexity 和中间矩阵存储，而不是把数学复杂度从 $O(S^2)$ 变成 $O(S)$ 。在现代 GPU 上，少搬数据能显著提升实际速度和内存效率。

Know-how：落到实现 / 训练系统

- 面试白板画 HBM↔SRAM 层级与 block loop。
- 验证时比较 HBM bytes、kernel time、峰值显存。
- 注意 causal/mask/dropout 与不同 head dim 的 kernel 支持。

深挖关键点 S 级题：回答中必须同时出现 exact、tiling、SRAM/HBM、online softmax 四个关键词。

高频追问

1. Online softmax 怎么保证数值等价？
2. 为什么不能一次把 K/V 都放 SRAM？
3. FlashAttention 后 FLOPs 还是 $O(S^2d)$ 吗？

常见失分点

- 说它是 sparse attention。
- 说它把复杂度改成 $O(S)$ 。

一句话记忆 FlashAttention 的创新中心是 IO-aware exact attention：少写 $S \times S$ 中间矩阵。

延伸阅读：R6 / R15

题 59 为什么减少理论 FLOPs 不一定减少 Wall-Clock？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察端到端性能建模。

3 分钟主答

真实 step time 由 compute、memory traffic、collective communication、kernel launch、同步、pipeline bubble、load imbalance 等共同决定。理论 FLOPs 只覆盖其中一部分。

例如 MoE active FLOPs 很低，但 token All-to-All 和小 expert GEMM 可能使 MFU 下降；某些 recompute 虽增加 FLOPs，却能降低内存压力并提升更大 batch 的整体吞吐。

Know-how：落到实现 / 训练系统

- 用 profiler 按 compute/memory/comm/idle 分类，而不是凭 FLOP 猜。
- 比较等 batch 和等 memory 两种 benchmark。
- 最终以 tokens/s、cost/token 作为系统指标。

高频追问

1. 为什么 kernel fusion 可能 FLOPs 不变但更快？
2. checkpointing 何时可能反而提升吞吐？
3. MoE 为什么是典型反例？

常见失分点

- 用理论 FLOPs 直接预测加速比。

一句话记忆 端到端优化对象是时间和成本；FLOPs 只是一个解释变量。

延伸阅读：R9

题 60 什么是 MFU？MFU 低应该怎样定位？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

训练系统日常指标，考察从症状到根因的能力。

3 分钟主答

Model FLOPs Utilization 用模型实际完成的理论 FLOPs/sec 除以加速器峰值 FLOPs/sec，反映端到端训练距离硬件计算上限有多远。

MFU 低不是结论。可能来自通信、pipeline bubble、dataloader、small GEMM、recompute、MoE imbalance、CPU 同步或 kernel 不匹配。需要用 profiler 将 step time 分解。

Know-how：落到实现 / 训练系统

- 先确保 FLOPs 口径与硬件峰值 dtype 对应。
- 记录 tokens/s、GPU active、NCCL time、bubble、dataloader wait。
- 从最大 time bucket 开始优化，避免为了 MFU 美化指标。

高频追问

1. MFU 与 HFU 有什么差别？
2. 为什么 checkpointing 可能让 MFU 定义看起来下降？
3. MFU 高是否说明训练成本一定低？

常见失分点

- 把 MFU 当 GPU utilization 百分比。

一句话记忆 MFU 是体检报告，不是诊断书；低 MFU 后面必须跟 profiler。

延伸阅读：R9

第 7 章 · 题 61-70

Mixture-of-Experts

从稀疏激活到路由、负载均衡、专家并行与 All-to-All。

本章面试策略

先把核心公式 / Shape 写出来，再解释设计动机；最后一定落到显存、通信、吞吐或数据分布。

题 61 MoE 为什么能“总参数很大、每 Token FLOPs 较小”？

B 类等价追问题

难度 易

频率 ★★★★★

面试官为什么问

MoE 基础，但必须说清 total vs activated parameters。

3 分钟主答

MoE 将 dense FFN 替换为多个 experts 和 router。对每个 token，router 只选择 Top-k experts 执行，因此模型可以拥有很大的 total capacity，而单 token 只激活少量 expert parameters。

Attention、embedding 等部分通常仍是 dense，因此 active FLOPs 不是简单用“激活参数/总参数比例”缩放全部网络。系统上还要付 routing 与通信成本。

Know-how：落到实现 / 训练系统

- 参数报告同时写 total params、active params/token、experts、top-k。
- 统计真实 tokens/expert 与 active FLOPs。
- 比较 dense baseline 时对齐总训练成本。

高频追问

1. MoE 是否天然更快？
2. 为什么 expert 主要替换 FFN？
3. Top-k routing 是否可微？

常见失分点

- 把 total params 当每 token compute。

一句话记忆 MoE 用稀疏激活解耦“容量”和“每 token 计算”，但通信并不会随 FLOPs 自动消失。

延伸阅读：R10

题 62 Top-1 与 Top-2 Routing 有什么 Trade-off?

B 类等价追问问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察质量、计算、鲁棒性与负载均衡。

3 分钟主答

Top-1 每 token 只进入一个 expert，计算和通信更低，但路由错误或热点 expert 的影响更直接；Top-2 允许两个 expert 共同贡献，提高组合表达与鲁棒性，但 active compute、dispatch bytes 和负载管理成本上升。

没有绝对优劣，最佳 k 取决于 expert granularity、routing quality、集群网络与目标成本。

Know-how：落到实现 / 训练系统

- 对齐 active FLOPs 比较不同 top-k。
- 观察 expert entropy、load balance、token drop 与 wall-clock。
- 用 quality-vs-cost Pareto，而非只看 loss。

高频追问

1. Top-2 权重如何归一化？
2. Top-k 越大是否趋近 dense？
3. routing noise 有什么作用？

常见失分点

- 说 Top-2 一定效果更好。

一句话记忆 Routing 的 k 是能力与系统成本的直接旋钮。

延伸阅读：R10

题 63 为什么 MoE 会出现 Expert Collapse / Hot Expert?

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

检验你是否理解路由正反馈及系统后果。

3 分钟主答

某个 expert 初期略占优势后会获得更多 token，从而收到更多梯度、进一步变强，形成 rich-get-richer；其他 experts 训练不足，容量被浪费。

系统层面不均衡会让某些 rank 计算更久、buffer overflow 或 token drop，整个同步 step 被最慢 expert 拖住。因此这是算法质量与系统吞吐的共同问题。

Know-how：落到实现 / 训练系统

- 监控每 expert token count、router probability、entropy、overflow 和 per-rank time。
- 区分语义 specialization 与无意义负载倾斜。
- 使用 balancing mechanism 后做质量/吞吐双指标 ablation。

深挖关键点 S 级题：把“路由动力学 → 梯度分配 → 负载 → 集群尾延迟”完整讲出来。

高频追问

1. 为什么完全均匀也未必最优？
2. collapse 与 expert specialization 怎么区分？
3. 如果只某个 domain 触发热点怎么办？

常见失分点

- 只把 collapse 当模型质量问题，不看 straggler。

一句话记忆 Expert balance 的目标不是强制相同，而是避免路由正反馈把容量和硬件都浪费掉。

延伸阅读：R9 / R10

题 64 Load-Balancing Auxiliary Loss 怎么工作？为什么太强会伤模型？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

从经典 MoE 平衡机制追到目标冲突。

3 分钟主答

典型辅助项根据 router probability 和实际 token assignment 的分布，惩罚专家使用过度不均，使 token 更平均地分散。它作为 $\lambda L_{\text{balance}}$ 加到主语言模型 loss。

若 λ 太大，router 会为了“均匀”牺牲语义最优选择：本应集中到某些专业 expert 的 token 被强制分散。因此 load balance 与 representation specialization 存在目标冲突。

Know-how：落到实现 / 训练系统

- 同时曲线化 LM loss、balance loss、expert utilization、吞吐。
- sweep λ 并检查 domain specialization。
- 避免只看全局均衡，按 batch/domain 检查局部热点。

高频追问

1. Balance loss 如何构造可微信号？
2. 为什么 expert 使用完全均匀不是最终目标？
3. capacity factor 与 balance loss 如何交互？

常见失分点

- 只说“辅助 loss 防止 collapse”，说不出副作用。

一句话记忆 平衡 loss 在优化硬件利用率，但不能压过语言建模的语义路由目标。

延伸阅读：R10

题 65 DeepSeek-V3 的 Auxiliary-Loss-Free Load Balancing 为什么重要？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

连接公开技术报告与 MoE 目标冲突，是当前预训练高价值题。

3 分钟主答

DeepSeek-V3 报告提出 auxiliary-loss-free 的负载均衡思路：不再依赖较强额外 balance loss 去扭曲主目标，而通过路由侧的 bias/调节机制促进 expert 负载平衡。

核心 Know-why 是把“系统需要均衡”与“模型需要语义最优路由”尽量解耦。它不是“不需要任何平衡控制”，而是改变平衡信号进入优化的方式。

Know-how: 落到实现 / 训练系统

- 讲清策略前先讲传统 auxiliary loss 的 gradient interference。
- 监控 bias 调整后 utilization 与 LM quality 是否同步改善。
- 报告仍可能保留的 sequence-wise/局部平衡约束，避免绝对化。

深挖关键点 S 级回答顺序：传统方案 → 目标冲突 → 路由侧调节 → 质量/系统双收益。

高频追问

1. 没有辅助 loss，平衡信号从哪里来？
2. bias 调节会不会影响最终路由？
3. 这种方法怎样应对突然的 domain shift？

常见失分点

- 把 “aux-loss-free” 说成 “完全没有负载均衡”。

一句话记忆 它的创新点是减少平衡目标对 LM 梯度的干扰，而不是放弃平衡。

延伸阅读：R10

题 66 Shared Expert 的意义是什么？

B 类等价追问

难度 中

频率 ★★★★★☆

面试官为什么问

考察 MoE 中通用知识与专业化的职责分配。

3 分钟主答

Shared expert 对所有 token 都激活，可承担通用变换；routed experts 则按 token 选择，负责更细的专业化。这样可减少每个 routed expert 重复学习通用模式的压力。

但 shared expert 增加固定 active compute，太大又会削弱稀疏优势。是否采用、容量多大需要按质量与 FLOPs 验证。

Know-how: 落到实现 / 训练系统

- 分开统计 shared/routed expert 的 FLOPs 与参数。
- 用表示/路由分析检查 routed experts 是否真的更专业。
- 比较相同 active FLOPs 下有无 shared expert。

高频追问

1. Shared expert 会不会成为新的 dense bottleneck？
2. 多个 shared experts 如何设计？
3. 为什么通用知识不能完全由 routed experts 学？

常见失分点

- 把 shared expert 说成 “永远更好”。

一句话记忆 Shared expert 用固定 compute 承担公共能力，换取 routed expert 更充分的 specialization。

延伸阅读：R10

题 67 Fine-Grained Experts 为什么可能优于少量大 Experts?

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察 expert granularity、组合表达与 kernel 效率。

3 分钟主答

将一个大 FFN expert 拆成更多较小 experts，并在相同 active parameter budget 下选择多个，可提供更丰富的组合空间和 specialization granularity。

代价是每个 expert GEMM 变小，GPU 利用率下降，routing metadata 与 All-to-All 更复杂。因此算法上的细粒度必须依靠 grouped GEMM、token batching 等系统优化才能兑现。

Know-how：落到实现 / 训练系统

- 对齐 active params 与 FLOPs 做 granularity ablation。
- profile expert GEMM size 和 Tensor Core efficiency。
- 监控 tokens/expert 是否足以形成大 batch。

高频追问

1. 为什么小 expert 更容易 communication-bound?
2. Grouped GEMM 解决什么?
3. expert 数增大如何影响 router 参数?

常见失分点

- 只谈“专家越多越专业”。

一句话记忆 Fine-grained MoE 是“组合表达更强”与“小 GEMM/通信更差”的典型 trade-off。

延伸阅读：R10 / R9

题 68 Expert Parallel 与 Tensor Parallel 如何相互影响?

B 类等价追问题

难度 难

频率 ★★★★★★

面试官为什么问

顶级 MoE 系统题，考察多维并行不是独立旋钮。

3 分钟主答

EP 已经把不同 experts 放到不同 rank；若 expert MLP 内再用很大的 TP，单个 expert 的矩阵会进一步被切小，可能导致 GEMM 利用率下降，同时增加 collective。

Attention 与 MoE 层的最佳 TP degree 可能不同。现代训练框架因此支持更灵活的 parallel folding/不同模块并行映射，目标是让 attention 大矩阵和 expert GEMM 都保持硬件友好。

Know-how：落到实现 / 训练系统

- 分别 profile attention 与 expert block 的 compute/comm。
- 从 GEMM shape 反推最小高效 local matrix。
- 让 EP/TP group 映射与节点拓扑一致。

高频追问

1. 为什么 MoE 常减少 expert TP？
2. EP×TP 能否随意相乘？
3. Attention TP 与 expert TP 不同怎么实现？

常见失分点

- 把所有并行度看成互不影响的乘法因子。

一句话记忆 多维并行真正的难点是：每一次“再切一刀”都会改变本地矩阵大小和通信频率。

延伸阅读：R9

题 69 MoE 为什么特别容易 Communication-Bound？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

考察稀疏 FLOPs 与网络数据移动的矛盾。

3 分钟主答

每个 token 在 expert 层前后需要按 router 结果重排并跨设备发送 hidden states。若专家小、top-k 多、EP 跨节点或 token 分布不均，All-to-All 时间可能接近甚至超过 expert GEMM。

因此减少 active FLOPs 不保证 wall-clock 下降。MoE 系统优化常围绕 topology-aware EP、grouped GEMM、dispatch fusion、communication overlap 与负载均衡。

Know-how：落到实现 / 训练系统

- 把 expert layer 时间拆成 route/dispatch/GEMM/combine。
- 计算 bytes/token 与 FLOPs/token 的通信计算比。
- 测试 intra-node vs inter-node EP。

高频追问

1. 怎样 overlap A2A 与 expert compute？
2. 为什么 token permutation 本身也有成本？

常见失分点

- 只看专家 FLOPs 判断加速比。

3. 哪些模型规模更适合 MoE?

一句话记忆 MoE 的“稀疏”省的是算术，不一定省网络；网络常成为新的墙。

延伸阅读：R9 / R10

题 70 Qwen3 MoE 与 DeepSeek-V3 MoE 应该怎样专业比较?

A 类高频真题型 难度 难 频率 ★★★★★

面试官为什么问

公开面经直接出现类似比较题，考察能否超越参数背诵。

3 分钟主答

比较框架应至少包括：total/activated params、expert 数与 top-k、expert granularity、shared experts、router/负载均衡、attention 架构、训练目标与系统通信。Qwen3 官方资料披露其 MoE 版本使用 128 experts、每 token 激活 8；DeepSeek-V3 报告为 671B total、约 37B active，并强调 DeepSeekMoE、aux-loss-free balancing 与 MTP。

不要只背“谁有多少专家”。架构差异最终要落到 capacity、active FLOPs、routing quality、A2A pressure、training stability 与 serving cost。

Know-how：落到实现 / 训练系统

- 先做统一字段比较表，避免不同报告口径混杂。
- 所有数字注明具体 model variant。
- 将算法差异映射到可观测系统指标。

深挖关键点 这类“模型对比题”不是记忆题；面试官真正看你能否建立统一分析坐标系。

高频追问

1. 为什么 expert 数不能直接表示 capacity?
2. 两者负载均衡思路有何不同?
3. Attention 结构会怎样影响整体 active compute?

常见失分点

- 混淆不同 Qwen3 variant。
- 只比较总参数和 benchmark。

一句话记忆 比较 MoE 的正确顺序：容量 → 激活 → 路由 → 平衡 → 通信 → 实际成本。

延伸阅读：R10 / R11 / R12 / R14

第 8 章 · 题 71-80

长上下文与高效 Attention

RoPE 外推、FlashAttention、Sliding Window、Context Parallel 与有效上下文评测。

本章面试策略

先把核心公式 / Shape 写出来，再解释设计动机；最后一定落到显存、通信、吞吐或数据分布。

题 71 Self-Attention 为什么是 $O(S^2)$?

B 类等价追问题

难度 易

频率 ★★★★★☆

面试官为什么问

长上下文所有优化的起点。

3 分钟主答

QK^T 需要计算 S 个 query 与 S 个 key 的两两相似度，score matrix 为 $S \times S$ ；若 head dimension d ，则主要 attention matmul FLOPs 为 $O(S^2d)$ ，naive score/probability 存储为 $O(S^2)$ 。

注意 Transformer 总 FLOPs 还包含 $O(Sd^2)$ 的 projection/MLP。当 d 很大、 S 较短时线性项可能占主导；长 S 时二次项才逐渐成为关键。

$$QK^T: [S, d] \times [d, S] \rightarrow [S, S]$$

Know-how：落到实现 / 训练系统

- 比较不同 S 下 attention 与 MLP FLOPs crossover。
- 不要把 $O(S^2)$ 直接等同整个模型总复杂度。
- 使用 profiler 验证真实瓶颈。

高频追问

1. Sequence 翻 2 倍，哪些项翻 2 倍/4 倍？
2. GQA 会改变 QK FLOPs 吗？
3. FlashAttention 改的是哪部分复杂度？

常见失分点

- 说整个 Transformer 永远 $O(S^2)$ 。

一句话记忆 Attention pair 数是 S^2 ，但模型总成本由 S^2d 与 Sd^2 共同决定。

延伸阅读：R1

题 72 FlashAttention 解决的是计算复杂度还是 IO Complexity?

A 类高频真题型

难度 易

频率 ★★★★★

面试官为什么问

经典陷阱题，用于验证是否真正理解论文。

3 分钟主答

FlashAttention 不改变 exact dense attention 的渐进数学 FLOPs：QK 和 PV 仍是 $O(S^2d)$ 。它通过 IO-aware tiling 减少 HBM 与片上 SRAM 之间的数据搬运以及 $S \times S$ 中间矩阵存储。

因此它常同时带来更快速度和更低显存，但不能据此声称 128K attention 已变成线性复杂度。

Know-how：落到实现 / 训练系统

- 回答中明确区分 arithmetic complexity、IO complexity、memory footprint。
- 端到端比较相同序列长度与 kernel 支持。

- 长 context 仍需 CP/稀疏 attention 等手段解决规模压力。

高频追问

1. 为什么 memory footprint 可以降低而 FLOPs 不降？
2. FlashAttention 是否会重算部分 score？
3. 它与 sliding-window attention 能叠加吗？

常见失分点

- 把 “memory efficient” 误写成 “O(S) attention”。

一句话记忆 FlashAttention 改执行计划，不改 dense attention 的数学定义。

延伸阅读：R6

题 73 RoPE 为什么会有 Length Extrapolation 问题？

B 类等价追问问题

难度 难

频率 ★★★★★

面试官为什么问

从位置公式追到训练分布外泛化。

3 分钟主答

RoPE 的不同维度对应不同旋转频率。训练只覆盖 $\text{position} \leq L_{\text{train}}$ 时，模型只见过一定范围内的相位组合和相对距离；推理把位置扩到远超训练范围，会出现未见过的频率相位关系和 attention pattern。

因此“公式可以计算任意 position”不等于“模型对任意长度有效”。需要 frequency/position scaling 与长序列 continued training 共同解决。

Know-how：落到实现 / 训练系统

- 查看不同频率维度在扩展位置上的相位变化。
- 长 context 评测按 needle depth/distance 分桶。
- 避免只测短任务拼成长 prompt。

高频追问

1. 高频/低频维分别更关注什么距离？
2. 为什么简单线性 position scaling 会损害短距离？
3. base theta 改变什么？

常见失分点

- 把 `max_position_embeddings` 改大就叫扩上下文。

一句话记忆 可计算的位置范围不是可泛化的上下文范围。

延伸阅读：R2

题 74 NTK-Aware Scaling / YaRN 本质上在做什么？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察长上下文 RoPE extension 的统一理解。

3 分钟主答

这类方法的共同目标是调整 RoPE 的位置/频率映射，让模型在更长 sequence 上不要骤然进入完全陌生的相位区域，同时尽量保留训练长度内的局部位置分辨率。

不同频率承载的距离尺度不同，所以简单把所有位置统一压缩往往会牺牲局部关系；更精细方法会对频率区间做差异化处理，并配合长文本训练。

Know-how：落到实现 / 训练系统

- 不必死背所有公式，先解释“局部精度 vs 长距外推”矛盾。
- 对短 context benchmark 做回归，防止 extension 伤原能力。
- 用多长度 validation loss/needle test 画曲线。

高频追问

1. Linear interpolation 的缺点？
2. 为什么要区分频率段？
3. 只做 RoPE scaling 不训练能走多远？

常见失分点

- 把 YaRN/NTK-aware 当成魔法扩大 context 的配置项。

一句话记忆 长上下文 RoPE scaling 是在“保局分辨率”和“扩远距离相位范围”之间做频率重映射。

延伸阅读：R2

题 75 为什么 Long-Context 通常还需要 Continued Pretraining?

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

区分位置可表示性与模型真正利用长上下文的能力。

3 分钟主答

修改 RoPE 只解决“位置如何编码”的一部分问题。模型若从未在长文档上训练，就未必学会跨数万 token 检索、聚合、维持主题或处理远距离依赖。

因此常在预训练后期加入长序列阶段，改变 context length 和长文档数据分布，并用较谨慎 LR 继续训练。Qwen3 官方资料也描述了阶段化预训练与长上下文阶段。

Know-how：落到实现 / 训练系统

- 构造真实长文档与合成长距依赖混合数据。

- 从短到长逐步扩展并监控通用能力回归。
- 长序列阶段同步调整 batch tokens、CP 与 checkpointing。

高频追问

1. 为什么只训练长序列的短文拼接不够？
2. CTP 会不会造成短上下文遗忘？
3. 长文本数据从哪里来？

常见失分点

- 把“支持 128K”只归功于 RoPE 参数。

一句话记忆 位置外推让模型“看得到”，长上下文训练让模型“学会用”。

延伸阅读：R11 / R12

题 76 Context 从 4K 扩到 32K，为什么常采用阶段式训练？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察训练分布、显存和优化稳定性的联动。

3 分钟主答

Sequence 从 4K 直接跳到 32K 会同时改变 attention/activation 成本、每 step 样本数、长文档分布和有效 global batch。若保持显存不变，microbatch 往往下降，优化噪声也随之变化。

阶段式 4K→8K→16K→32K 可逐步暴露数值与系统问题，让 RoPE/长文能力和并行配置一起迁移；但阶段数并非固定，需要按资源和稳定性实验决定。

Know-how：落到实现 / 训练系统

- 每个阶段按 consumed tokens 重新检查 LR、batch token 与 validation。
- 提前做各长度的 memory/throughput capacity plan。
- 在长度切换点保存 checkpoint 和完整 telemetry，便于归因。

高频追问

1. 如果算力足够能否直接 32K？
2. global batch token 应保持不变吗？
3. 长序列阶段为什么更容易 data starvation？

常见失分点

- 把 curriculum 当成必然更优，不做等 token 对照。

一句话记忆 扩上下文不仅改 position；它同时改变数据、优化和系统三套分布。

延伸阅读：R11

题 77 Sliding-Window Attention 的优缺点是什么？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察用稀疏性换长上下文成本的基本思路。

3 分钟主答

若每个 token 只关注局部窗口 w ，attention pair 从 S^2 降到约 Sw ，能显著降低长序列计算和显存。局部语言建模本身也有很强归纳偏置。

代价是单层无法建立任意远距离直接连接；全局信息只能靠多层传播、周期性 global attention 或特殊 token。对于精确长距离检索可能成为能力瓶颈。

$$\text{Attention pairs} \approx S \times w, w \ll S$$

Know-how：落到实现 / 训练系统

- 设计 local/global layer pattern 并测长距 retrieval。
- 同时观察短上下文 perplexity 与真实长文 QA。
- kernel 是否真正利用 sparsity 决定墙钟收益。

高频追问

1. 窗口 w 应怎么定？
2. 多层局部 attention 的有效感受野多大？
3. 它与 FlashAttention 是否冲突？

常见失分点

- 只用理论 $O(Sw)$ 断言一定更快。

一句话记忆 Sliding window 用稀疏连接换成本；最核心风险是远距离信息路径变长。

题 78 用了 FlashAttention 为什么长上下文训练仍然 OOM？

A 类高频真题型

难度 中

频率 ★★★★★★

面试官为什么问

检验是否把 FlashAttention 错当成“消灭所有 activation”。

3 分钟主答

FlashAttention 主要消除/重排 $S \times S$ attention 中间存储，但 Transformer 仍有 residual stream、QKV、MLP 中间 activation、norm/dropout、通信 buffer 等，许多张量仍随 $B \times S \times H \times L$ 线性增长。

因此 128K 等极长训练通常还需要 activation checkpointing、Context Parallel、microbatch 调整和 selective recompute。

Know-how: 落到实现 / 训练系统

- 用 memory snapshot 找峰值 tensor，不根据印象优化。
- 逐项测试 FA、checkpointing、CP 的增益与开销。
- 注意 allocator fragmentation 与临时 workspace。

高频追问

1. 哪类 activation 不会被 FlashAttention 消掉？
2. CP 为什么能继续降显存？
3. sequence length 翻倍后剩余 activation 怎么变？

常见失分点

- 认为用了 FA 就不需要 checkpointing。

一句话记忆 FlashAttention 解决 attention matrix IO；长序列剩余 activation 仍按序列长度增长。

延伸阅读：R6 / R9

题 79 Context Parallel 下 Attention 是怎么计算的？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

考察长上下文分片后的全局 Attention 正确性。

3 分钟主答

CP 将 sequence 维分到多个 ranks，每个 rank 持一部分 query/context。为了让本地 Q 仍能注意全局 key/value，需要在 ranks 间交换 K/V（常见 ring 或 all-gather 类方案），逐块累积 attention 结果与 softmax 统计。

它降低每卡长序列 activation，但增加 forward/backward 的 context communication；CP degree 越高，本地 sequence 越短，同时网络占比上升。

Know-how: 落到实现 / 训练系统

- 画 rank0/rank1 的 Q-local、KV-rotating 数据流。
- 关注 causal mask 在分块/环式顺序中的正确性。
- 同时 profile memory gain 与 network time。

深挖关键点 高分回答要同时说清“切了什么、为了数学等价必须交换什么、换来了什么显存”。

高频追问

1. CP 与 sequence parallel 为什么不同？
2. Online softmax 怎样支持分块 KV？
3. CP 是否改变模型数学结果？

常见失分点

- 只说“把 sequence 切开”，没有解释跨块 attention。

一句话记忆 CP 的核心矛盾：Q 可以本地，但 exact attention 所需 K/V 必须跨 rank 流动。

延伸阅读：R9

题 80 如何证明一个“128K 模型”真的有 128K 有效能力？

B 类等价追问题

难度 难

频率 ★★★★★☆

面试官为什么问

考察评测设计，防止把配置长度当模型能力。

3 分钟主答

最大可输入长度只是接口能力。有效长上下文应测试不同距离/位置的检索、多跳聚合、长文档 QA、全局一致性，以及真实文档而非单一 synthetic needle。

还要控制“答案是否可由局部窗口猜出”、prompt position bias 和 benchmark contamination；最好绘制 accuracy 随 context length、needle depth、distance 的二维曲线。

Know-how：落到实现 / 训练系统

- 构造短/中/长分桶与不同插入深度。
- 同时评估短上下文回归，确保扩长没有伤基础能力。
- 测真实长文档与合成 diagnostic 两套集。

高频追问

1. Needle-in-a-haystack 有什么缺陷？
2. 如何测试模型聚合多个远距证据？
3. 128K perplexity 低就够了吗？

常见失分点

- 只跑一个 needle case。
- 把能接受 128K input 当成能利用 128K。

一句话记忆 “支持长度”是系统接口；“有效长度”必须由距离敏感、不可局部作弊的任务证明。

第 9 章 · 题 81-90

训练稳定性、Debug 与评测

真正做过预训练的人，必须能定位 Loss Spike、NaN、Straggler、数据异常与恢复问题。

本章面试策略

先定义可观测指标，再讨论实验与归因；避免只给经验结论。

题 81 100B 模型训练到 42k Step 突然 Loss Spike，怎么排查？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

最能区分“读过论文”和“真正训过模型”的故障题。

3 分钟主答

第一原则是定位与保全证据，而不是马上降 LR。先确定第一个异常 step/rank，保存 batch IDs、数据来源、loss、grad norm、activation statistics、optimizer state 与 NCCL/硬件日志。然后按 Data / Numerical / Optimizer / Distributed / Hardware 五类建立假设。

若有 spike 前 checkpoint，使用同一 batch deterministic replay。可复现则更偏数据/forward/numerical；不可复现或多个 batch 持续异常，则检查 optimizer state、并行同步和硬件。修复后用小范围 replay/回归证明根因。

Know-how：落到实现 / 训练系统

- 建立自动 anomaly snapshot：异常 step 把 batch hash 与关键 tensor stats 落盘。
- 先恢复到 spike 前 checkpoint 做受控复现，避免在昂贵 full run 上试错。
- 每次只改变一个变量：skip batch、lower LR、disable node、换数据 shard。

深挖关键点 S 级题：回答必须体现 incident response，而不是列一串可能原因。

高频追问

1. 如何区分 bad batch 与 optimizer instability?
2. Grad norm 大但 loss 正常怎么办?
3. 如果只有一个 rank 先 NaN 呢?

常见失分点

- 第一反应“调低学习率/加 clipping”。
- 没有保存异常 batch，导致无法复现。

一句话记忆 Loss spike 的专业处理是“冻结现场 → 可复现 → 单变量实验 → 根因 → 回归”，而不是经验调参。

题 82 如何区分 Bad Batch 与 Optimizer Instability?

B 类等价追问题

难度 难

频率 ★★★★★

面试官为什么问

故障定位中最实用的因果实验题。

3 分钟主答

从 spike 前 checkpoint 固定 RNG 与数据顺序，重放异常 batch。若每次都在相同位置出现异常，而换正常 batch 恢复，则强烈支持 data/forward 特异问题；若不同 batch 都逐步失稳，说明参数/optimizer state 已进入不稳定区域。

还可将异常 batch 送入较早 checkpoint，或将正常 batch 送入临近 checkpoint，形成 2×2 交叉实验，把“数据因素”和“状态因素”解耦。

Know-how: 落到实现 / 训练系统

- 保存 batch hash/source/length/token stats。
- 做 checkpoint×batch 交叉矩阵并比较 loss/gradnorm。
- 确定是否 deterministic，排查 fused kernel/硬件随机故障。

高频追问

1. 异常 batch 一定要删除吗？
2. 如果 bad batch 只在大模型上炸怎么办？
3. 如何定位 batch 内具体 sample？

常见失分点

- 仅凭“这一 step 数据看起来怪”下结论。

一句话记忆 最强诊断工具是交叉 replay：把数据和模型状态拆开做因果测试。

题 83 Gradient Norm 突然变大意味着什么？

B 类等价追问问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察是否会从单一指标跳到联合诊断。

3 分钟主答

Grad norm 升高可能来自真正的 gradient explosion，也可能是一个罕见高损失 batch、sequence/domain 切换、router imbalance、activation scale drift 或某层数值异常。单点指标不能直接给根因。

应联合看 loss、per-layer grad norm、activation RMS、parameter/update norm、data metadata，并判断异常是全局还是集中在某层/某 expert。

Know-how: 落到实现 / 训练系统

- 保存 global + layerwise grad stats。
- 对 spike step 做 top-k layer/module 排名。
- 检查 clipping 前后 norm，避免 clipping 隐藏长期异常。

高频追问

1. Grad clipping 应设多大？
2. loss 不变但 gradnorm 变大可能是什么？
3. 如何定位是哪一层贡献最大？

常见失分点

- 看到 grad norm 高就立即加大 clipping。

一句话记忆 Grad norm 是烟雾报警器；先定位哪间房起火。

题 84 Overall Loss 在降，但 Math/Code Benchmark 不涨，为什么？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

考察平均 loss 掩盖 domain 能力的现象。

3 分钟主答

总体 token loss 被高占比、较易 domain 主导。若 Web 通用文本占绝大多数，哪怕 math/code 的 loss 停滞，总 loss 仍可能稳定下降；离散 benchmark 还存在阈值与格式影响。

正确做法是维护 domain-conditioned validation loss、能力 proxy 和数据 consumed tokens，观察目标域边际收益。如果目标能力停滞，再判断 mixture、数据难度、tokenizer 或模型容量。

Know-how：落到实现 / 训练系统

- 为 math/code/语言分别建 held-out validation。
- 对每个阶段记录 mixture 与实际 sample consumption。
- 结合 loss 与 benchmark，避免只盯一个。

高频追问

1. domain loss 能直接跨域比较吗？
2. 增加 math 比例会伤通用能力吗？
3. 如何判断是数据不够还是模型容量不够？

常见失分点

- 因为 benchmark 不涨就立刻加数据比例。

一句话记忆 总体 loss 是加权平均；预训练能力必须按目标 domain 拆开看。

题 85 Training/Validation Loss 都下降，就能说明模型更好吗？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察评测集代表性与 contamination。

3 分钟主答

若 validation 与训练分布过近、存在重复/污染，loss 同降只说明在这套分布上拟合更好；它不保证长尾、推理、代码、事实性或安全等能力同步提升。

预训练成功需要 held-out clean loss、domain-specific eval、capability benchmark、memorization/contamination audit 与效率指标共同支撑。

Know-how：落到实现 / 训练系统

- 验证集独立 source/time split，避免网页镜像泄漏。

- 保留多个 domain 和难度层级。
- 评测版本与数据版本绑定。

高频追问

1. Validation 数据应该多大？
2. 时间切分有什么价值？
3. 如何检测 validation 被 dedup cluster 泄漏？

常见失分点

- 把 validation loss 当唯一 KPI。

一句话记忆 Loss 是训练的基础健康指标，不是完整的能力证明。

题 86 如何监控 1000+ GPU 的预训练任务？

B 类等价追问问题

难度 难

频率 ★★★★★

面试官为什么问

考察 observability、分层指标与自动恢复。

3 分钟主答

至少分四层：模型层看 loss/grad/update/activation；数据层看 tokens/sec、domain、length、shard；GPU 层看利用率、HBM、温度/ECC；分布式层看 NCCL time、rank step time、bubble、straggler。

规模越大，平均指标越危险。必须保留 per-rank/percentile 视图与异常关联，才能发现“1 张卡拖慢 1000 张”的尾部问题。

Know-how：落到实现 / 训练系统

- 建立 step-level trace id，把 data batch、checkpoint、rank 与日志关联。
- 对 p50/p95/p99 step time 和 NCCL time 告警。
- 自动 snapshot + safe checkpoint + restart policy。

高频追问

1. 哪些指标必须 per-rank？
2. 日志太多如何降采样？
3. 如何检测 silent data corruption？

常见失分点

- 只看 GPU utilization 和全局 loss。

一句话记忆 千卡可观测性的核心是“全局趋势 + rank 尾部 + 数据血缘”能关联到同一 step。

延伸阅读：R9

题 87 单个 GPU 比其他 GPU 慢 20%，整个 Job 会怎样？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

考察同步训练的 straggler 放大效应。

3 分钟主答

同步训练每个 collective/step 都要等待最慢参与者，故 step time 接近关键路径上的最大 rank 时间。单卡持续慢 20% 可能使整个数据并行组乃至全局流水线明显降速。

原因可能是硬件降频、ECC、NUMA/PCIe、网络链路、数据不均、某 expert 热点等。要先确定慢在 compute、communication 还是 input。

$$T_{\text{step}} \approx \max_i T_i \text{ (同步关键路径近似)}$$

Know-how：落到实现 / 训练系统

- 画 per-rank step-time heatmap。
- 对异常节点做同 workload A/B swap。
- 在 MoE 中同时查 token/expert imbalance。

高频追问

1. 为什么平均 GPU util 看不出来？
2. 一个 PP stage 慢与一个 DP rank 慢有何区别？
3. 是否应该自动剔除慢卡？

常见失分点

- 只重启 job 而不定位节点/网络。

一句话记忆 同步系统的速度由尾部决定；千卡训练必须把 straggler 当一等公民。

延伸阅读：R9

题 88 训练出现 NaN，你的排查顺序是什么？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

训练稳定性必问，要求从第一个坏 tensor 逆推。

3 分钟主答

先定位第一个出现 NaN/Inf 的 step，再用 hooks/统计确定是 forward 还是 backward、哪个 layer、哪个 tensor 首先异常。随后检查输入数据、attention logits/softmax、norm、loss、gradient、optimizer update 和低精度 scale。

关键是找到“第一个 NaN”，不是最后崩掉的 loss。必要时回到前一 checkpoint，关闭部分 fused/低精度路径做二分复现。

Know-how：落到实现 / 训练系统

- 开启 finite-check 只在短 replay 中精细定位，避免 full run 巨大开销。
- 逐层记录 max/min/RMS 和 finite ratio。
- 修复后用异常 batch + 正常 batch 双回归。

高频追问

1. NaN 首先出现在 softmax 可能是什么？
2. BF16 也会 NaN 吗？
3. 为什么 torch.nan_to_num 不是修复？

常见失分点

- 直接在 loss 上 nan_to_num。
- 只查最后一层。

一句话记忆 NaN 调试的目标是找到第一个非法数值的出生地，而不是清理尸体。

题 89 Checkpoint 应保存什么，才能做到真正可恢复？

B 类等价追问题

难度 难

频率 ★★★★★

面试官为什么问

考察预训练的状态机，而非只保存模型权重。

3 分钟主答

完整恢复通常需要模型参数、optimizer states、LR scheduler、RNG states、data loader/shuffle state、consumed tokens/step、并行拓扑与 scaler 等。否则 resume 后虽然权重一样，后续数据和随机性可能不同。大规模 checkpoint 还涉及分片格式、异步写、原子提交、校验和与跨并行度加载。可恢复性必须做定期演练。

Know-how：落到实现 / 训练系统

- 采用 manifest + checksum，最后原子标记 checkpoint 完成。
- 训练中定期做 disaster-recovery drill。
- 保存数据 cursor/seed，验证 resume 后若干 step 可复现。

高频追问

1. 改变 DP/TP 后能否加载？
2. 异步 checkpoint 如何保证一致性？
3. 只保存 consumed step 为什么不够？

常见失分点

- checkpoint 文件存在就认为可恢复。

一句话记忆 Checkpoint 的定义是“完整训练状态的可验证快照”，不是一堆权重文件。

题 90 如何定义一个预训练模型“训练成功”？

B 类等价追问题

难度 中

频率 ★★★★★☆

面试官为什么问

开放题，考察目标函数以外的完整成功标准。

3 分钟主答

至少包含五类：optimization 稳定收敛；capability 达到目标 benchmark/held-out loss；generalization 覆盖关键 domain；efficiency 满足 tokens/s、MFU、成本预算；reliability 没有不可接受的污染、记忆与恢复风险。

“成功”必须相对项目目标和 compute budget 定义。一个 loss 更低但多花 2 倍成本、推理难部署的模型未必是更成功的产品。

Know-how：落到实现 / 训练系统

- 项目立项时预先写 success criteria，而不是训练后挑指标。
- 同时保留质量、成本、风险三个 dashboard。
- 做 baseline 与同预算比较。

高频追问

1. 能力与成本冲突时怎么选？
2. 什么时候应该提前停止训练？
3. 如何定义 data-quality success？

常见失分点

- 只回答“benchmark 更高”。

一句话记忆 预训练成功 = 质量达标、预算可控、系统可靠、结果可信。

第 10 章 · 题 91-100

手撕、系统设计与项目拷打

把“知道”变成“能写、能算、能设计、能复盘、能证明”。

本章面试策略

先定义可观测指标，再讨论实验与归因；避免只给经验结论。

题 91 手写 Multi-Head Attention，面试官真正看什么？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

公开面经高频手撕题，shape 正确比 API 熟练更重要。

3 分钟主答

输入 $[B, S, D]$ 经过 QKV projection, reshape/transposition 为 $[B, H, S, D_h]$; 计算 $\text{score} = [B, H, S, S]$, 除 $\sqrt{D_h}$, 加入 causal/padding mask, softmax 后与 V 相乘, 再 transpose/reshape 回 $[B, S, D]$ 并做 output projection。

面试官通常观察你是否能持续维护 shape、softmax 维度、mask 语义和 contiguous/view 问题，而不是能否背 torch.nn.MultiheadAttention。

$[B, S, D] \rightarrow [B, H, S, D_h] \rightarrow \text{scores } [B, H, S, S] \rightarrow [B, S, D]$

Know-how: 落到实现 / 训练系统

- 先在白板逐行标 shape，再写代码。
- mask 使用足够负值并考虑 dtype。
- 为 $B=2, S=4, H=2$ 写最小单元测试。

高频追问

1. padding mask 与 causal mask 怎么合并？
2. GQA 如何改代码？
3. KV Cache 版 decode 如何写？

常见失分点

- softmax dim 写错。
- reshape 前后 head/sequence 维混乱。

一句话记忆 手撕 Attention 的第一原则：每一行代码旁边都知道 tensor shape。

延伸阅读：R13 / R16

题 92 手写 Causal Mask：如何保证语义和数值都正确？

B 类等价追问题

难度 易

频率 ★★★★★☆

面试官为什么问

小题但能暴露 off-by-one 与 dtype 错误。

3 分钟主答

对 query 位置 i ，只允许访问 key 位置 $j \leq i$ ；未来位置加 $-\infty$ （或 dtype 安全的极小值）后再 softmax。若包含 padding，还要把 padding key 同时屏蔽。

在 packed sequence 中，单纯下三角 mask 可能不够：如果不允许跨文档 attention，还要加入 document boundary mask。

$$M_{ij}=0 \text{ if } j \leq i \text{ else } -\infty$$

Know-how: 落到实现 / 训练系统

- 用 4×4 手工矩阵做 unit test。
- 检查 diagonal 是否可见、padding query 的 loss 是否 mask。
- 低精度下避免不安全常数导致 overflow/NaN。

高频追问

1. 为什么 mask 加在 softmax 前?
2. packed docs 如何 block diagonal?
3. FlashAttention 的 causal flag 等价什么?

常见失分点

- 把未来位置设 0 再 softmax。
- 搞反上三角/下三角。

一句话记忆 Causal mask 不是“把未来概率清零”，而是让未来 logit 在 softmax 前不可竞争。

题 93 手写 RMSNorm: 最常见错误是什么?

A 类高频真题型

难度 易

频率 ★★★★★☆

面试官为什么问

用十几行代码判断是否真的懂统计量和数值类型。

3 分钟主答

RMSNorm 计算 $\text{mean}(x^2)$ 的平方根，不减均值，也不是 $x.\text{std}()$ 。通常在最后一维归一化，再乘可学习 scale γ 。

低精度实现常把平方和/mean 转到 FP32 累积，再 cast 回输入 dtype，以降低 reduction 误差。

$$y = x / \sqrt{(\text{mean}(x^2) + \epsilon)} \cdot \gamma$$

Know-how: 落到实现 / 训练系统

- 测试输入全常数向量，区分 RMSNorm 与 LayerNorm。
- 检查 epsilon 和维度。
- 与框架 reference implementation 做 max error 对照。

高频追问

1. 为什么全常数输入 LayerNorm 与 RMSNorm 结果不同?
2. weight 要用什么 dtype?

常见失分点

- 误用 std。
- 忘记 epsilon。

3. 能否 fused 到其他 kernel?

一句话记忆 RMSNorm = RMS 尺度归一化，不做中心化。

题 94 手写 RoPE：代码之外必须证明什么？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

手撕位置编码时，数学与实现要互相验证。

3 分钟主答

实现通常把 head dimension 两两配对，用对应 position/frequency 的 $\cos$ 、 $\sin$ 做二维旋转，再应用到 Q/K。核心正确性不仅是 shape，还应证明旋转保持向量范数，并说明 QK 点积如何依赖相对位置。

工程中要特别检查 position_ids、不同 batch offset、KV Cache decode 位置以及长上下文频率缩放。

$$R(\theta)[x_1, x_2]^T = [x_1 \cos \theta - x_2 \sin \theta, x_1 \sin \theta + x_2 \cos \theta]^T$$

Know-how：落到实现 / 训练系统

- 用小向量验证旋转前后 L2 norm 相同。
- 数值验证相同相对距离在适当条件下产生一致结构。
- decode 时测试 position offset，避免 cache 位置错位。

高频追问

1. rotate_half 有哪些实现约定？
2. 为什么只对 Q/K 不对 V？
3. GQA 下 RoPE 如何广播？

常见失分点

- 只抄 $\cos/\sin$ 代码，不能解释相对位置。

一句话记忆 RoPE 手撕要同时过三关：二维旋转、shape/broadcast、相对位置推导。

延伸阅读：R2

题 95 手算一个 Transformer Block 的参数量怎么做？

B 类等价追问题

难度 中

频率 ★★★★★

面试官为什么问

参数量是架构设计、FLOPs 和显存手算的共同基础。

3 分钟主答

标准 MHA 中 Q/K/V/O 四个 $d \times d$ 矩阵约 $4d^2$ ；传统 4d FFN 两矩阵约 $8d^2$ ，所以经典 block 粗约 $12d^2$ ，忽略 bias/norm。

现代模型用 GQA 与 SwiGLU 时必须重新算：K/V projection 由 h_{kv} 决定；SwiGLU 有 gate/up/down 三矩阵，intermediate dimension 通常不再简单等于 $4d$ 。

经典 MHA+4d FFN: $P_{\text{attn}} \approx 4d^2$, $P_{\text{ffn}} \approx 8d^2$, $P_{\text{block}} \approx 12d^2$

Know-how: 落到实现 / 训练系统

- 先写每个 linear 的 $\text{in} \times \text{out}$ ，再相加。
- 最后乘 layers，加 embedding/LM head。
- 所有近似明确是否 weight tying、bias。

高频追问

1. GQA 如何改变 QKV 参数？
2. SwiGLU 为什么不能仍按 $8d^2$ ？
3. embedding 对小模型为什么占比更高？

常见失分点

- 无论架构都背 $12d^2$ 。

一句话记忆 参数手算不要背总式：把每个 projection 的矩阵尺寸列出来，就不会错。

延伸阅读：R1

题 96 手算 70B + Adam + BF16 的训练显存，并推到 ZeRO

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

预训练岗位 S 级题：假设、公式和分片必须都对。

3 分钟主答

先声明一种常见假设：BF16 params 2B、BF16 grads 2B、FP32 master 4B、Adam m/v 各 4B，总计 16 bytes/param。70B model states 约 1.12TB；这还不包括 activation、临时 buffer 和 fragmentation。

然后按 ZeRO stage 分片：Stage1 只把 optimizer 部分按 DP 分；Stage2 再分 grad；Stage3 连 param 也按 DP 分。真正单卡峰值还要加入参数 all-gather、activation 和通信 workspace。

示例基线： $70 \times 10^9 \times 16 \text{ bytes} \approx 1.12 \text{ TB}$ (model states, 未分片)

Know-how: 落到实现 / 训练系统

- 白板用表格列 state、dtype、总字节、是否 /DP。
- 给出“理论常驻”与“运行峰值”两个数字。
- 说明不同框架没有 master copy 时数字会变化。

深挖关键点 S 级题必须练到任何参数规模、DP degree 和 ZeRO stage 都能现场推。

高频追问

1. DP=64, ZeRO-2 常驻 model states 多少?
2. Stage3 为什么峰值不等于总状态/DP?
3. 加 activation checkpointing 影响哪一项?

常见失分点

- 不声明 dtype 假设。
- 把全部 16B/param 在 ZeRO-1 就除以 DP。

一句话记忆 显存手算模板：列状态 → 算 bytes → 标复制/分片 → 加 activation/峰值。

延伸阅读：R7 / R9 / R15

题 97 从 0 预训练一个 1B 模型，完整 Pipeline 如何设计?

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

综合检验 architecture、data、training 与 evaluation 的闭环。

3 分钟主答

先定义目标能力/语言/上下文与预算；再设计 tokenizer 和模型；构建可追溯数据 pipeline 与 mixture；用小规模 scaling/recipe experiments 定 LR、batch、数据策略；完成单卡→多卡 correctness/stability tests 后再 full run。

训练期间持续做 held-out loss、domain eval、system telemetry、checkpoint/recovery；结束后进行 contamination、memorization、能力和效率评估。重点不是列流程，而是每个阶段都有退出条件。

Know-how: 落到实现 / 训练系统

- 建立 1M→10M→100M token smoke test 阶梯。
- 每一阶段只有在 loss、numerics、resume、data correctness 通过后扩大 GPU。
- 先定义 rollback 与 checkpoint 策略再开大任务。

深挖关键点 这道题应回答成一套“stage-gate 体系”，而不是流水账。

高频追问

1. 最先做哪三个实验?

常见失分点

- 直接从 64/128 GPU full run 开始。

2. 如何确定 1B 该训多少 token?
3. 什么情况下你会终止 full run?

- 流程没有验收门槛。

一句话记忆 大规模预训练的工程原则：让错误尽可能在更少 GPU、更少 token 时暴露。

延伸阅读：R5 / R9 / R11

题 98 1000 张 GPU 给你一个月，怎样降低 Full Run 失败概率？

B 类等价追问题

难度 难

频率 ★★★★★

面试官为什么问

考察可靠性工程与昂贵实验的风险管理。

3 分钟主答

把失败风险前移：在小规模完成 data/tokenizer unit tests、distributed correctness、resume determinism、mixed-precision stability、throughput benchmark；再按 GPU 数和 token 数逐级扩大。

Full run 中要有多副本/异步 checkpoint、异常快照、节点健康监控、自动重启与明确的“继续/回滚”策略。还应保留部分算力预算用于 pilot 与突发恢复。

Know-how：落到实现 / 训练系统

- 建立 preflight checklist：loss parity、gradient parity、data uniqueness、resume parity。
- chaos test：模拟节点失败与 checkpoint 恢复。
- 训练期间定期恢复最近 checkpoint 跑短验证。

高频追问

1. 最危险的 silent failure 是什么？
2. checkpoint 多久存一次最优？
3. 如何评估存 checkpoint 的 IO 成本？

常见失分点

- 把稳定性等同于“多存 checkpoint”。

一句话记忆 降低大模型失败概率的本质，是把未知风险转成小规模可测试、可观测、可恢复的问题。

延伸阅读：R9

题 99 面试官问“你真的从 0 预训练过模型吗？”怎样证明？

A 类高频真题型

难度 中

频率 ★★★★★

面试官为什么问

项目真实性拷打，不靠术语靠可核对细节。

3 分钟主答

真正做过的人应自然给出：参数规模、训练 token、GPU 型号/数量、sequence length、global batch tokens、peak LR/warmup、optimizer、precision、TP/PP/DP/CP/EP、MFU/tokens/s、checkpoint cadence、最终 validation loss。

更有说服力的是讲一个真实事故：什么时候 OOM/loss spike/straggler，如何定位、改了什么、吞吐/质量变化多少。具体数字和因果链比“熟悉 Megatron”有力得多。

Know-how：落到实现 / 训练系统

- 提前为自己项目做一张 one-page fact sheet。
- 所有指标准备“定义与口径”，防止追问时自相矛盾。
- 区分团队工作与个人负责模块。

高频追问

1. 你亲手改了哪段 pipeline？
2. MFU 怎么算的？
3. 如果重来一次你会改什么？

常见失分点

- 只列技术栈。
- 把团队成果全部说成个人成果。

一句话记忆 项目可信度来自可核对的数字、失败案例和个人决策，而不是工具名。

题 100 你在预训练项目中真正解决的最难问题是什么？

A 类高频真题型

难度 难

频率 ★★★★★

面试官为什么问

终局题：检验研究闭环、工程深度、所有权和量化结果。

3 分钟主答

推荐用 Observation → Hypothesis → Experiment → Root Cause → Fix → Ablation → Quantitative Result 讲述。先给异常现象和业务/训练成本，再列 2-3 个竞争假设；说明你用什么 profile/replay/对照实验排除它们，最后给根因。

修复后必须有回归：质量是否保持？tokens/s、MFU、显存、故障率改善多少？若只是“改了配置然后好了”，说明因果证据仍不够。

Know-how：落到实现 / 训练系统

- 提前准备至少三个故事：数据、训练稳定性、系统性能各一个。
- 每个故事控制在 3 分钟主线，追问时再展开技术细节。
- 量化 before/after，并说明实验控制变量。

深挖关键点 建议把这一题录音练 10 次：任何“我们觉得/应该是”都替换成可观测证据或受控实验。

高频追问

1. 你如何证明这就是 root cause?
2. 有没有更便宜的替代方案?
3. 这个经验能迁移到更大模型吗?

常见失分点

- 讲成流水账，没有竞争假设。
- 只有“结果更好”，没有等成本/回归实验。

一句话记忆 最强项目回答不是“我解决了什么”，而是“我怎样证明为什么它坏、为什么这个修复有效”。

附录 A | 面试现场答题模板

架构题 定义/公式 → 旧方案瓶颈 → 关键设计 → 参数/FLOPs/KV → Trade-off → 可验证实验。

分布式题 切哪个维度 → 每 rank 持什么 → 什么 collective → 显存降多少 → 通信加多少 → 如何映射拓扑。

显存题 声明 dtype/optimizer → 列 bytes/param → 算全局 → 应用 TP/DP/ZeRO/FSDP → 单独估 activation/临时峰值。

故障题 定位第一异常 step → 保存 batch/rank/tensor → replay → 交叉实验 → root cause → fix → 回归验证。

项目题 Observation → Hypothesis → Experiment → Root Cause → Fix → Ablation → Quantitative Result。

附录 B | 四周冲刺计划

阶段	范围	训练目标
Week 1	01-30	架构、Tokenizer、数据、Optimizer；每天 8-10 题，强制白板写公式。
Week 2	31-60	Scaling + 分布式 + 显存；每天至少 2 道手算 + 1 张通信图。
Week 3	61-90	MoE、长上下文、Debug；每天做一个“故障树”口述演练。
Week 4	91-100 + S级复盘	手撕、系统设计、项目拷打；录音模拟面试并按 Why/How/Trade-off/Production 打分。

每天 90 分钟最小训练单元

- 1. 20 分钟：闭卷口述 5 题。
- 2. 25 分钟：白板推 1 个公式 + 算 1 道显存/并行题。
- 3. 25 分钟：把 1 道题映射到自己的真实项目。
- 4. 20 分钟：只记录“为什么卡住”，不抄整段答案。

参考资料与题源说明

技术部分优先使用原始论文、官方技术报告和训练框架文档；公开面经仅用于识别招聘中反复出现的题型，不视为公司官方题库。

R1 Vaswani et al., Attention Is All You Need
<https://arxiv.org/abs/1706.03762>

R2 Su et al., RoFormer: Rotary Position Embedding
<https://arxiv.org/abs/2104.09864>

R3 Ainslie et al., GQA
<https://arxiv.org/abs/2305.13245>

R4 Shazeer, GLU Variants Improve Transformer
<https://arxiv.org/abs/2002.05202>

R5 Hoffmann et al., Training Compute-Optimal Large Language Models (Chinchilla)
<https://arxiv.org/abs/2203.15556>

R6 Dao et al., FlashAttention
<https://arxiv.org/abs/2205.14135>

R7 Rajbhandari et al., ZeRO
<https://arxiv.org/abs/1910.02054>

R8 Shoeybi et al., Megatron-LM
<https://arxiv.org/abs/1909.08053>

R9 NVIDIA Megatron Core User Guide / Parallelism / MoE
<https://docs.nvidia.com/megatron-core/developer-guide/latest/>

R10 DeepSeek-AI, DeepSeek-V3 Technical Report
<https://arxiv.org/abs/2412.19437>

R11 Qwen Team, Qwen3 Technical Report
<https://arxiv.org/abs/2505.09388>

R12 Qwen3 Official Blog
<https://qwenlm.github.io/blog/qwen3/>

R13 公开面经：字节大模型算法岗，含 FSDP/DDP/Megatron、OOM/Activation Checkpointing、MHA
<https://www.nowcoder.com/discuss/922308546966847488>

R14 公开面经：阿里大模型算法岗，含 Packing/Padding、Qwen3、MoE 对比、Transformer 手撕
<https://www.nowcoder.com/feed/main/detail/15ac418ca4394f1b97edbfbd00a51cda>

R15 公开面经：京东大模型算法实习，含 Qwen、DeepSpeed/DDP/FlashAttention、显存、GQA
<https://www.nowcoder.com/feed/main/detail/e4ac3528bcdd44ebb8c3617540216932>

R16 公开面经：爱奇艺大模型算法岗，含通用预训练流程、长文本训练、BF16/FP32、MHA
<https://www.nowcoder.com/feed/main/detail/f35c2be602e44be09637c974102c11f3>

版权说明

本手册为独立编写学习资料。“参考《剑指 Offer》”仅指借鉴问题驱动、层层追问、强调思路与现场表达的学习范式，不复制其原文、题解或版式。

把“知道”练成“能证明”。

Know-What → Know-Why → Know-How → Trade-off → Production

LLM 预训练算法岗面试 100 题 · 2026